

QUASAR: Lowering the Loss Floor of Quantization-Aware Training with Loss-Aware Reconstruction

Vincent Counathe^{1,2}

Ben Athiwaratkun²

Christopher De Sa^{1,2}

Tianyi Zhang²

¹Cornell University ²Together AI

August 13, 2026

Abstract

As large language model inference shifts toward lower precision, post-training quantization (PTQ) becomes increasingly brittle, making quantization-aware training (QAT) essential for preserving model quality. However, QAT computes the loss and surrogate gradients using a lossy reconstruction of latent full-precision weights, while applying updates to the latent weights themselves. This mismatch can lead to suboptimal training trajectories and a higher loss floor. Second-order PTQ methods mitigate a similar gap by minimizing loss-aware reconstruction error, but doing it once for a frozen model can take hours; repeating this process throughout QAT as the weights evolve is impractical. We introduce QUASAR, a QAT method that continuously performs lightweight, loss-aware reconstruction in the training loop to lower the loss floor and improve the resulting low-bit model. At each training step, QUASAR uses the exponential moving average of squared gradients as online saliency estimates, searches over a small set of clipping ranges, and fits affine dequantizers via saliency-weighted least squares. Our analysis shows that the loss-aware reconstruction error is the only reconstruction-dependent term in the QAT convergence bound and controls the loss of the final quantized model, establishing QUASAR’s objective as a principled optimization target. QUASAR modifies only the training procedure and supports standard deployment formats, including integer quantization and NVFP4, with no inference-time changes or overhead. Across Qwen3 and Llama-3.1, QUASAR achieves the lowest held-out KL divergence among competitive QAT methods at 2, 3, and 4 bits, reducing KL by at least 10% at 3 and 4 bits and by 29% at 2 bits. At 2 bits, it improves average accuracy across eight tasks by 3.5–4.3 percentage points over strong QAT and PTQ baselines. Applied to NVFP4, QUASAR reduces held-out KL by approximately 30% relative to standard QAT. Finally, under direct low-bit adaptation for mathematical reasoning, QUASAR outperforms both QAT and full-precision training followed by PTQ by at least 10.9 percentage points across five math benchmarks.

1 Introduction

As large language models (LLMs) grow in size and adoption, inference is overtaking training as the dominant recurring cost of deployment [9, 54]. To reduce this cost, inference is rapidly moving toward low-precision formats such as FP4 and INT4 [16, 48, 52], which shrink the model’s memory footprint, reduce decoding latency, and enable higher concurrency and throughput [37]. However, since models are typically trained in high precision but served in low precision [18], preserving model quality as precision becomes increasingly aggressive remains a critical challenge. Post-training quantization (PTQ) has been the standard way to convert a trained model to low precision, but it

is becoming increasingly brittle on newer models. Reasoning and agentic models often operate over long contexts, where quantization errors can accumulate and degrade performance [33, 39, 44]. PTQ models may also exhibit behavioral shifts and biases relative to their full-precision counterparts [12, 41, 42]. Producing a native low-precision model with stronger quality guarantees therefore requires training under the same precision used for serving.

Quantization-aware training (QAT) addresses this need by inserting quantization into the training loop, allowing the model to adapt to quantization noise as it learns [3]. QAT, however, introduces a structural mismatch into optimization: the forward pass and loss use quantized–dequantized reconstruction weights r , while the optimizer updates the latent full-precision weights w . Because quantization has zero gradient almost everywhere, QAT typically uses the straight-through estimator (STE), approximating $\frac{\partial L}{\partial w}$ with $\frac{\partial L}{\partial r}$. This surrogate is generally not the optimal descent direction for the latent weights, leading training along a suboptimal trajectory. The consequence is a *loss-floor gap*: given the same model and training data, QAT converges to a higher final training and held-out loss than full-precision training [6, 28]. Prior QAT methods narrow this gap by softening the rounding operation, learning quantization parameters jointly with the weights, or improving initialization [11, 14, 19]. We instead draw on a complementary insight from second-order PTQ methods: the excess training loss caused by QAT’s structural mismatch can be reduced by optimizing the reconstruction process.

Second-order PTQ methods such as GPTQ [16] offer a useful perspective on the structural mismatch in QAT. A second-order expansion of the loss around the full-precision weights w , assuming that the first-order term is negligible near an optimum, gives

$$L(r) - L(w) \approx \frac{1}{2}S, \quad S = (r - w)^\top H(r - w),$$

where r denotes the quantized–dequantized reconstruction and H is the Hessian evaluated at w . We refer to S as the *loss-aware reconstruction error*. Many modern PTQ methods [5, 15, 16, 32, 46] minimize this quantity, or an approximation to it, to construct a low-precision model whose loss remains close to that of a frozen full-precision model. Applying the same principle to QAT would keep the low-precision loss $L(r)$ closer to the full-precision loss $L(w)$ as the latent weights evolve, thereby mitigating the effect of the structural mismatch. The challenge, however, is computational cost. In PTQ, the full-precision model is frozen, so its reconstruction needs to be optimized only once, typically requiring minutes to hours for a billion-parameter LLM. In QAT, however, the latent weights change at every step, requiring the reconstruction to be continually re-optimized. Running a conventional second-order optimization procedure on every forward pass would therefore be prohibitively expensive.

We introduce **QUASAR** (**Q**uantization-aware training with **l**oss-aware **r**econstruction), a QAT method that continually minimizes loss-aware reconstruction error to improve the training trajectory and lower the loss floor. QUASAR decomposes reconstruction into two stages: *quantization*, which maps full-precision weights to discrete codes while treating the clipping range as a free parameter, and *dequantization*, which maps those codes back to reconstructed weights using a learned scale and, for asymmetric quantization, an optional offset. To make the loss-aware objective tractable during training, QUASAR approximates the Hessian using an exponential moving average of squared gradients, yielding per-parameter saliency scores that estimate each weight’s effect on the loss. It then searches over a small set of candidate clipping ranges, each defining a different assignment of discrete codes; for each assignment, it solves for the dequantization parameters in closed form by saliency-weighted least squares and selects the candidate with the smallest loss-aware reconstruction error. Our theoretical analysis decomposes the QAT convergence bound

into three terms: initialization, minibatch noise, and loss-aware reconstruction error. Only the last depends on the reconstruction map, and it is exactly the objective QUASAR minimizes at each step. Reconstruction is therefore a direct lever on the training trajectory and, under an additional PL condition, on the loss of the final quantized model (Section 5).

Empirically, QUASAR consistently achieves lower training and evaluation loss than competitive QAT baselines across bit widths and model families. For quantization-aware distillation of Qwen3-4B-Thinking [58], QUASAR achieves lower final evaluation loss than Standard QAT, LSQ [14], Denoising QAT [59], and BitDistiller [11] at every tested bit width, reducing evaluation loss by at least 10% at INT4 and INT3 and by at least 29% at INT2. The gains extend to NVFP4, where QUASAR reduces evaluation loss by about 30% relative to Standard QAT while also improving downstream accuracy. We further apply QUASAR to supervised fine-tuning of the Qwen3-4B Base model at INT4/3/2 using reasoning traces from OpenMathReasoning [45], where QUASAR outperforms competitive QAT and PTQ baselines by at least 10.9 points in average accuracy across five math benchmarks at INT2. These improvements come at little additional cost: QUASAR increases training step time by only about 1.5% and introduces no inference overhead.

2 Related Work

In this section we review the two lines of work QUASAR sits between: PTQ and QAT. PTQ treats quantization as a one-shot algorithm: given a trained model and a small calibration set, select a low-precision value for each weight that damages the loss least. Concretely, most methods preserve quality of the trained model using a second-order expansion. For instance, Optimal Brain Surgeon quantifies a perturbation by its curvature-weighted magnitude [21], OBQ and GPTQ carry it to LLMs by quantizing one column at a time and updating the not-yet-quantized weights to absorb the error introduced by quantized weights [15, 16], and AdaRound, BRECQ, or YAQA learn the rounding itself against a reconstruction objective [32, 46, 57]. All these methods rest on the premise that the weights are frozen and final, hence the reconstruction happens only once.

A second PTQ line of work expands what a one-shot quantization may be. For instance, QuIP, QuIP#, QTIP, and AQLM spread outliers with random orthogonal transforms and replace the quantization grid with lattice, trellis, or additive codebooks [5, 13, 55, 56]. At two and three bits these are the strongest PTQ methods available, however, they require a Hadamard transform or a codebook decode at inference, negatively impacting throughput, while hardware has instead been moving toward simpler block-scaled formats such as MXFP4 and NVFP4 [48, 52].

Below four bits, the PTQ methods that do not add inference overhead degrade sharply. QAT achieves higher quality at these bit widths through various methods: LSQ learns the step size [14], LLM-QAT and BitDistiller supervise with the full-precision model [11, 40], and QLoRA freezes the quantized weights to train low-rank adapters over them [10]. What none of them do is optimize the map from latent weights to reconstructed weights, which is mostly set by heuristics.

QUASAR directly optimizes that map. At every step it searches over clipping ranges and fits the dequantizer by saliency-weighted least squares, so the reconstruction PTQ solves once is effectively brought into the training loop as weights adapt. Section 5 shows that the resulting error controls the reconstruction-induced gradient mismatch and, through it, the loss of the final quantized model. The search and fit produce only a scale and offset, so the trained model deploys exactly as an RTN-quantized one.

3 Preliminaries

3.1 Reconstruction in Standard QAT

Standard QAT stores the full-precision latent weights w in memory and updates them during training. However, the forward pass and loss computation use a different set of weights: the quantized-dequantized reconstructed weights r . These reconstructed weights are transient and are derived from the current latent weights for each forward pass. For each weight group, the reconstruction process first maps the latent weights to integer codes, $w \rightarrow q$, and then maps those codes back to real-valued weights, $q \rightarrow r$. The forward pass and loss are computed using r , while the optimizer updates w (Figure 1).

The mapping from w to q involves rounding, which has no useful derivative. QAT therefore uses the STE [3]:

$$\frac{\partial L}{\partial w} \approx \frac{\partial L}{\partial r}.$$

In asymmetric integer quantization, each row of a weight matrix is partitioned into groups of g weights, such as $g=128$. A group $w \in \mathbb{R}^g$ is represented by b -bit codes $q \in \{0, \dots, q_{\max}\}$, where $q_{\max} = 2^b - 1$. All weights in the group share a scale s and zero-point z :

$$q_i = \text{clamp}\left(\left\lfloor \frac{w_i - z}{s} \right\rfloor\right), \quad r_i = s q_i + z. \quad (1)$$

Equation (1) defines two steps. First, *code assignment* maps each weight to an integer by shifting, scaling, rounding, and clamping. Second, *dequantization* maps the integer code to the reconstructed weight $r_i = s q_i + z$. The network uses r to compute the forward pass and loss.

A reconstruction requires two decisions. The first is how to assign integer codes q based on the clipping range. The second is how to select the dequantization parameters (s, z) that map these codes to reconstructed weight values. Standard QAT makes both decisions based only on the extreme weight values, without considering the loss. Specifically, it sets the clipping range to the minimum and maximum values in the group:

$$[z, z + s q_{\max}] = [\min_i w_i, \max_i w_i].$$

This choice places the extreme weights at the endpoints of the quantization grid. The resulting scale and zero-point are then reused for dequantization (Figure 2, top).

3.2 The Connection Between Weight Reconstruction and Training Loss in QAT

The model trains through the reconstructed weights r . The forward pass, loss, and gradient through the STE are all computed at r , while the optimizer applies the resulting update to the latent weights w . Therefore, errors introduced by the reconstruction can affect every optimization step.

At step t , let $\Delta_t := r_t - w_t$ and let $H_t := \nabla^2 \mathcal{L}(w_t)$ denote the full Hessian at the current latent weights. Throughout, $\|\cdot\|$ is the Euclidean norm for vectors and the induced ℓ_2 operator norm for matrices. We expand the neural network’s loss to second order around w_t . Near an optimum, the first-order term is negligible, giving $\mathcal{L}(r_t) - \mathcal{L}(w_t) \approx \frac{1}{2} S_t$, where

$$S_t = \Delta_t^\top H_t \Delta_t. \quad (2)$$

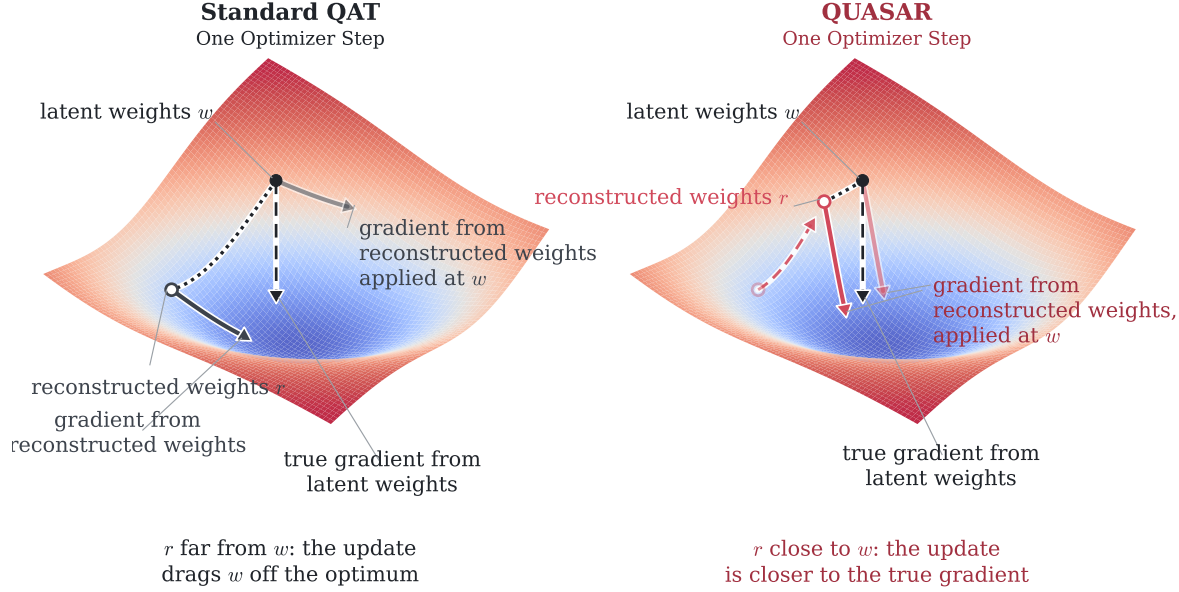


Figure 1: **Training through the reconstruction.** Under the STE, the gradient is computed at the reconstructed weights r and applied to the latent weights w at every step. QUASAR uses scale search and optimized dequantization parameters to keep r close to w . As a result, the gradient updates applied to the latent weights more closely approximate the gradients evaluated at those weights.

We refer to S_t as the *loss-aware reconstruction error*.¹ Section 5 shows that QUASAR’s tractable approximation $\hat{S}_t$ is the explicit reconstruction-dependent term in the QAT convergence bound and, under a PL condition, controls an upper bound on the loss of the final deployable model. Modern PTQ methods minimize the full-Hessian error, or a variant of it, once for a frozen model [16, 46, 57].

Applying this machinery within the QAT training loop is prohibitively expensive computationally. PTQ solvers rely on costly per-layer linear algebra. Such computations are practical as a one-time operation on a frozen model, although they can still take minutes or hours for models with billions of parameters [15, 16]. In QAT, the weights change after every update, so each solution immediately becomes stale. Repeating the same optimization at every training step would be prohibitively expensive.

In summary, every QAT method must produce a lossy reconstruction of the full-precision model within the constraints of a low-bit format, and the quality of this reconstruction directly affects the training loss. QUASAR builds on the insight that this reconstruction can be optimized by minimizing the approximation $\hat{S}_t$, reducing the harmful effect of reconstruction error on the training loss.

4 Methodology

In this section, we introduce QUASAR, a QAT method that applies loss-aware weight reconstruction throughout training to improve the quality of the final model. We describe its two core techniques: scale search and optimal dequantization. We then show how to extend QUASAR to

¹PTQ methods minimize variants of this quantity under different names and use different approximations to the Hessian [16, 46, 57].

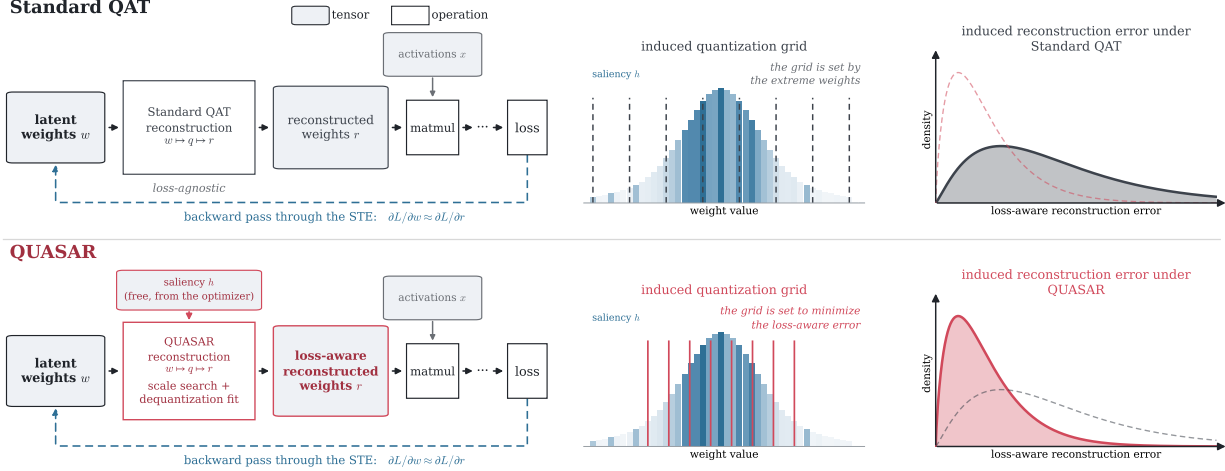


Figure 2: **Standard QAT vs. QUASAR.** The two training loops differ in only one component: the reconstruction, or the $w \rightarrow q \rightarrow r$ mapping. Standard QAT determines its quantization grid from the extreme weights in each group without considering the loss. QUASAR instead selects the grid to minimize the loss-aware reconstruction error. The right side shows the resulting quantization grids and error distributions. Appendix B provides empirical measurements.

production inference formats such as NVFP4.

4.1 QUASAR: Minimizing Reconstruction Error at Every Step

Bringing loss-aware reconstruction into the QAT training loop presents two challenges. First, computing the exact Hessian at every step is intractable. Second, even if the Hessian were available, finding the optimal reconstruction for a given set of full-precision weights would also be intractable. QUASAR addresses the first challenge with an online Hessian proxy that is often already available during training. It addresses the second with a lightweight procedure that optimizes quantization and dequantization together.

4.1.1 A Tractable Loss-Aware Objective

Computing the full Hessian at every training step is prohibitively expensive. Instead, we use the diagonal Fisher as a nonnegative proxy for curvature. We estimate it online using an exponential moving average of the squared gradients. Adam and AdamW already maintain this quantity as their second moment v_t [27]. When either optimizer is used, QUASAR can reuse this estimate without additional memory overhead.

We denote the estimate by h and approximate the Hessian as $H_t \approx \text{diag}(h)$. QUASAR then minimizes

$$S_t \approx \hat{S}_t(r; w, h) = \sum_i h_i (r_i - w_i)^2. \quad (3)$$

We call h_i the saliency of w_i . In a low-bit format, all g weights in a group share a single scale and zero-point. The reconstruction error must therefore be distributed across the group. Reducing the error for one weight may increase it for another. The saliencies h determine this tradeoff. QUASAR reconstructs high-saliency weights more accurately and allows larger errors for low-saliency weights. This allocation reflects each weight’s contribution to $\hat{S}_t$. By optimizing code assignment and de-

quantization together, QUASAR minimizes $\hat{S}_t$ and keeps the loss at the reconstructed weights close to the loss at the latent weights.

4.1.2 Optimizing Both Stages of Reconstruction

To find a low-error reconstruction that satisfies the quantization constraints, QUASAR optimizes both stages of the reconstruction: code assignment $w \mapsto q$ and dequantization $q \mapsto r$. During each forward pass, QUASAR evaluates several candidate clipping ranges. Each range produces a different code assignment. For each assignment, QUASAR computes the optimal dequantization parameters. It then selects the reconstruction with the lowest loss-aware reconstruction error.

A weighted least-squares fit gives the optimal dequantization. For a fixed code assignment q , which is induced by a candidate clipping range, QUASAR chooses the dequantization parameters (s, z) in $r_i = sq_i + z$ that minimize Equation (3). This is a weighted least-squares problem with the following closed-form solution:

$$s^* = \frac{\sum_i h_i (q_i - \bar{q}_h)(w_i - \bar{w}_h)}{\sum_i h_i (q_i - \bar{q}_h)^2}, \quad z^* = \bar{w}_h - s^* \bar{q}_h, \quad (4)$$

where

$$\bar{q}_h = \frac{\sum_i h_i q_i}{\sum_i h_i}, \quad \bar{w}_h = \frac{\sum_i h_i w_i}{\sum_i h_i}.$$

Thus, for any fixed codes q , QUASAR finds the dequantization parameters that minimize $\hat{S}_t$. Symmetric quantization admits a related closed-form solution.

Dequantization alone cannot correct a poor code assignment. Weights assigned the same code must have the same reconstructed value, regardless of the choice of (s, z) . This limitation becomes especially important at low bit widths, where few codes are available. QUASAR therefore combines optimal dequantization with scale search to improve the code assignment.

Scale search finds good code assignments. QUASAR searches over clipping ranges, which determine the code assignments in $w \mapsto q$. Each candidate scale factor $f \in \mathcal{F} \subseteq (0, 1]$ produces a code assignment q_f . For each assignment, QUASAR computes the optimal dequantization parameters using Equation (4). Let $\hat{S}_t(f)$ denote the resulting minimum error. QUASAR selects

$$f^* = \arg \min_{f \in \mathcal{F}} \hat{S}_t(f) = \arg \min_{f \in \mathcal{F}} \min_{s, z} \sum_i h_i (sq_{f,i} + z - w_i)^2.$$

The candidate clipping range determines only the code assignment. After the codes are assigned, the optimal dequantization parameters determine the reconstructed values. Scale search changes which weights share a code, while (s, z) determines the value represented by each code. Together, these choices minimize $\hat{S}_t$ over the candidate reconstructions at every step (Algorithm 1 and Figure 2, bottom).

Scale search is also effective empirically. During quantization-aware distillation of Qwen3-4B with 3-bit integer quantization, 99.6% of groups select a range narrower than the full minimum-to-maximum range. This selection reduces $\hat{S}_t$ to 69% of the full-range baseline (Appendix B, Figure 13).

Algorithm 1 QUASAR reconstruction. QUASAR searches over clipping ranges and, for each range, fits the scale and zero-point to minimize saliency-weighted reconstruction error.

Require: latent weights $w \in \mathbb{R}^g$; saliency $h \in \mathbb{R}_+^g$; grid of clipping-range factors $\mathcal{F} \subseteq (0, 1]$

- 1: $c \leftarrow \frac{1}{2}(\min_i w_i + \max_i w_i)$; $\rho \leftarrow \frac{1}{2}(\max_i w_i - \min_i w_i)$
- 2: **for** $f \in \mathcal{F}$ **do** $\triangleright$ scale search
- 3: $q_f \leftarrow$ codes of w on the range $[c - f\rho, c + f\rho]$ $\triangleright$ Eq. (1)
- 4: $(s_f, z_f) \leftarrow \text{DEQUANTPARAMS}(w, q_f, h)$ $\triangleright$ Eq. (4)
- 5: $\hat{S}_t(f) \leftarrow \sum_i h_i (s_f q_{f,i} + z_f - w_i)^2$ $\triangleright$ Eq. (3)
- 6: **end for**
- 7: $f^* \leftarrow \arg \min_{f \in \mathcal{F}} \hat{S}_t(f)$
- 8: **return** $r = s_{f^*} q_{f^*} + z_{f^*}$

When $\mathcal{F} = \{1\}$ and $h \equiv \mathbf{1}$, the algorithm optimizes only the dequantization parameters and reduces to Denoising QAT [59]. Standard QAT instead reuses the scale and zero-point of the full range ($f=1$).

QUASAR modifies only the weight reconstruction used during the forward pass. It does not change activation quantization or the backpropagation. Gradients pass through the weight quantizer using the same STE, and neither scale search nor dequantization fitting requires a separate backward rule.

4.2 Applying QUASAR to NVFP4

QUASAR is not limited to integer quantization. Its two reconstruction steps, searching for code assignments and fitting dequantization parameters, also apply to formats such as NVFP4. NVFP4 is a widely used 4-bit floating-point format for production inference [2].

The NVFP4 format. NVFP4 represents each weight on the symmetric E2M1 grid, which uses one sign bit, two exponent bits, and one mantissa bit. The format has two levels of scaling: one FP32 scale for the entire tensor and one FP8 scale for each group of 16 weights. Because NVFP4 is symmetric, it uses scale factors but no zero-point.

QUASAR for NVFP4. We set the tensor-level FP32 scale using the tensor’s absolute maximum and do not optimize it. Within each group, QUASAR searches over candidate scales to produce different code assignments q on the E2M1 grid. For each assignment, it selects the FP8 dequantization scale that minimizes the loss-aware objective in Equation (3). QUASAR then chooses the code assignment and fitted FP8 scale with the lowest loss-aware reconstruction error.

The search scale determines the E2M1 code assignment. The fitted FP8 scale determines the reconstructed values and is stored for inference. Thus, QUASAR retains the same objective and two-stage optimization used for integer quantization while satisfying the grid and scaling constraints of NVFP4. Section 6.4 presents the corresponding empirical results. Appendix B describes the extension to the Rubin LUT3 format.

5 Theoretical Analysis: Loss-Aware Reconstruction Bounds the Loss of the Final Quantized Model

In this section, we analyze how QUASAR’s reconstruction objective affects QAT optimization dynamics. QAT maintains latent full-precision weights w_t , while its forward passes use their low-bit reconstruction r_t . Under the identity STE, the gradient evaluated at r_t is applied to w_t , creating

the reconstruction-induced gradient mismatch

$$e_t := \nabla \mathcal{L}(r_t) - \nabla \mathcal{L}(w_t). \quad (5)$$

Prior work on biased-SGD theory shows that systematic gradient error introduces an additional convergence penalty [1]. Prior work has also studied when STE gradients provide useful descent directions [60] and how curvature affects QAT plateaus [31]. Our analysis shows that QUASAR’s tractable objective $\hat{S}_t$ is the only reconstruction-dependent term in the QAT convergence bound: minimizing it is therefore the precise lever through which reconstruction controls both the training trajectory (Theorem 1) and the loss of the final deployable model (Corollary 1).

5.1 Setup and Assumptions

Notation and weight reconstruction. Let $\mathcal{L} : \mathbb{R}^d \rightarrow \mathbb{R}$ be the training objective and $\mathcal{L}^* := \inf_x \mathcal{L}(x)$. At step t , let $w_t \in \mathbb{R}^d$ denote the latent full-precision weights, $h_t \in \mathbb{R}_+^d$ the saliencies, and $\mathcal{R}_t$ the finite set of candidate reconstructions considered by QUASAR. For any $r \in \mathcal{R}_t$, define

$$\Delta_t(r) := r - w_t, \quad \hat{S}_t(r) := \sum_i h_{t,i} \Delta_{t,i}(r)^2. \quad (6)$$

Here, $\Delta_t(r)$ is the reconstruction-induced weight perturbation, and $\hat{S}_t(r)$ is the loss-aware reconstruction error. QUASAR selects the reconstructed weights r_t as

$$r_t \in \arg \min_{r \in \mathcal{R}_t} \hat{S}_t(r). \quad (7)$$

Let $\mathcal{H}_t$ denote the training history before minibatch $\mathcal{B}_t$ is sampled. The candidate set and reconstruction are determined by $\mathcal{H}_t$, and we write $\mathbb{E}_t[\cdot] := \mathbb{E}[\cdot \mid \mathcal{H}_t]$.

We use the following assumptions. Assumptions 1–3 are sufficient for the main QAT convergence result, while Assumption 4 is only needed to control the final deployable model.

Assumption 1 (Objective regularity). The objective $\mathcal{L}$ is twice continuously differentiable, L -smooth, and bounded below:

$$\mathcal{L}^* = \inf_x \mathcal{L}(x) > -\infty.$$

Assumption 2 (Stochastic STE gradient). The identity-STE stochastic gradient can be written as

$$G_t = \nabla \mathcal{L}(r_t) + \xi_t, \quad \mathbb{E}_t[\xi_t] = 0,$$

and assuming

$$\mathbb{E}_t \|\xi_t\|^2 \leq M \|\nabla \mathcal{L}(r_t)\|^2 + \sigma^2 \quad (8)$$

for constants $M, \sigma^2 \geq 0$. The STE-SGD update we analyse is the following:

$$w_{t+1} = w_t - \eta G_t.$$

Assumption 3 (Hessian control). There exists a finite constant C such that, for every reconstruction step under consideration, every $r \in \mathcal{R}_t$, and every $u \in [0, 1]$,

$$\|\nabla^2 \mathcal{L}(w_t + u(r - w_t)) (r - w_t)\|^2 \leq C \hat{S}_t(r). \quad (9)$$

Assumption 4 (PL condition). For some $\mu > 0$, the objective satisfies the global Polyak–Łojasiewicz inequality

$$\|\nabla \mathcal{L}(x)\|^2 \geq 2\mu(\mathcal{L}(x) - \mathcal{L}^*) \quad \text{for all } x \in \mathbb{R}^d. \quad (10)$$

Remark 1 (Assumptions). The smoothness and stochastic-noise conditions in Assumptions 1 and 2 are standard in stochastic-optimization analysis. Assumption 1 requires the objective to be twice continuously differentiable and smooth, with a finite infimum (and no convexity). Assumption 2 allows gradient noise to grow with the gradient itself and generalizes the classical bounded-variance condition (recovered with $M = 0$); it is the same noise condition under which biased SGD is analyzed [1]. In words, Assumption 3 says the loss surface is well behaved along the few candidate directions QUASAR probes at each step: a reconstruction with small loss-aware error cannot trigger an outsized change in the gradient. A simple sufficient condition is a uniform positive saliency floor: if every saliency is at least $h_{\min} > 0$, then $\|\nabla^2 \mathcal{L}(y)(r - w_t)\|^2 \leq L^2 \|r - w_t\|^2 \leq (L^2/h_{\min}) \hat{S}_t(r)$, which is Assumption 3 with $C = L^2/h_{\min}$. The L -smoothness used here is standard in nonconvex SGD analysis [1, 4, 17]. The stronger condition $\nabla^2 \mathcal{L}(y)^2 \preceq C \text{diag}(h_t)$ would suffice, but Assumption 3 only requires control along QUASAR’s candidate segments. Adam’s second moment motivates the saliencies as a practical curvature proxy, but does not by itself guarantee this condition. Assumption 4 is the Polyak–Łojasiewicz (PL) condition, the standard route from stationarity to function-value control [26, 50]. It is needed only for Corollary 1, not for the convergence result of Theorem 1. PL is implied by strong convexity but does not require convexity: it allows nonconvex objectives whose minimizers form a large set, and it implies that every stationary point is a global minimum [26]. Local PL-type conditions have been established under specific overparameterized neural-network regimes [38].

Remark 2 (H versus H^2). The squared gradient mismatch $\|e_t\|^2$ is, to first order, $\Delta^\top H^2 \Delta$, so minimizing an H^2 -weighted error is also a natural objective: it targets the gradient mismatch rather than the loss gap. Theoretically, however, nothing is lost by using QUASAR’s H -weighted objective: it already controls the squared gradient mismatch up to the factor C (Theorem 1), so minimizing $\hat{S}_t$ keeps both the loss gap and the mismatch small. Empirically, we notice the diagonal H^2 proxy $\sum_i h_i^2 \Delta_i^2$ performs worse and is less stable than QUASAR’s objective (Appendix A.3, Figure 11).

5.2 From Reconstruction Error to QAT Convergence

5.2.1 Reconstruction Error Bounds the Gradient Mismatch

Our analysis has two parts. First, it connects reconstruction error to gradient mismatch, independently of the optimizer. Second, it shows how this mismatch enters the convergence guarantee for STE-SGD.

Theorem 1 (Reconstruction error affects gradient mismatch and convergence). *Under Assumptions 1–3, let $T \geq 1$. Then, for every $t < T$, QUASAR’s reconstruction-induced gradient mismatch satisfies*

$$\|\nabla \mathcal{L}(r_t) - \nabla \mathcal{L}(w_t)\|^2 \leq C \hat{S}_t(r_t) = C \min_{r \in \mathcal{R}_t} \hat{S}_t(r). \quad (11)$$

Moreover, if

$$0 < \eta < \frac{1}{L(1+M)},$$

then

$$\begin{aligned} \frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E} \left[\|\nabla \mathcal{L}(w_t)\|^2 + (1 - L\eta(1 + M)) \|\nabla \mathcal{L}(r_t)\|^2 \right] &\leq \frac{2(\mathcal{L}(w_0) - \mathcal{L}^*)}{\eta T} + L\eta\sigma^2 \\ &+ \frac{C}{T} \sum_{t=0}^{T-1} \mathbb{E} \hat{S}_t(r_t). \end{aligned} \quad (12)$$

The bound separates three terms: initialization, minibatch noise, and reconstruction error. The initialization term decreases with training time, while the noise term is the usual stochastic-optimization penalty. The remaining term is the penalty associated with weight reconstruction, which QUASAR minimizes at each step. Thus, lower reconstruction error gives a tighter guarantee that both the latent and reconstructed weights approach stationarity.

5.2.2 Comparison with Standard and Biased SGD

To compare the guarantees, define

$$B_T := \frac{\mathcal{L}(w_0) - \mathcal{L}^*}{\eta T} + \eta\sigma^2, \quad \bar{S}_T := \frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E} \hat{S}_t(r_t).$$

Up to constant factors, the relevant bounds are

$$\begin{aligned} \text{Standard SGD:} \quad & \frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E} \|\nabla \mathcal{L}(w_t)\|^2 \lesssim B_T, \\ \text{Biased SGD:} \quad & \frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E} \|\nabla \mathcal{L}(w_t)\|^2 \lesssim \frac{B_T + \zeta^2}{1 - m}, \\ \text{QUASAR:} \quad & \frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E} [\|\nabla \mathcal{L}(w_t)\|^2 + \|\nabla \mathcal{L}(r_t)\|^2] \lesssim B_T + C\bar{S}_T. \end{aligned} \quad (13)$$

Here, biased-SGD analysis assumes $\|b_t\|^2 \leq m\|\nabla \mathcal{L}(w_t)\|^2 + \zeta^2$ with $0 \leq m < 1$ [1]; in QAT, this bias is the gradient mismatch $b_t = e_t$. Biased SGD treats the bias as given; in QAT it comes from the reconstruction map, which we control. Our bound therefore replaces the fixed allowance ζ^2 with the realized error $C\bar{S}_T$, which QUASAR minimizes at every step, and it holds at both w_t and the deployed weights r_t . The rate is standard; the penalty term is now something training can act on.

5.3 Connecting Reconstruction Error to Final Quantized Loss

The previous theorem shows how reconstruction error affects convergence during training. For deployment, however, what matters is the loss of the final quantized reconstruction r_T , not only the stationarity of the latent weights. Under an additional PL condition, the next result connects the two by providing a direct bound on the final model's loss.

Corollary 1 (Final quantized model). *Under Assumptions 1–4, suppose*

$$0 < \eta < \min \left\{ \frac{1}{L(1 + M)}, \frac{1}{\mu} \right\}.$$

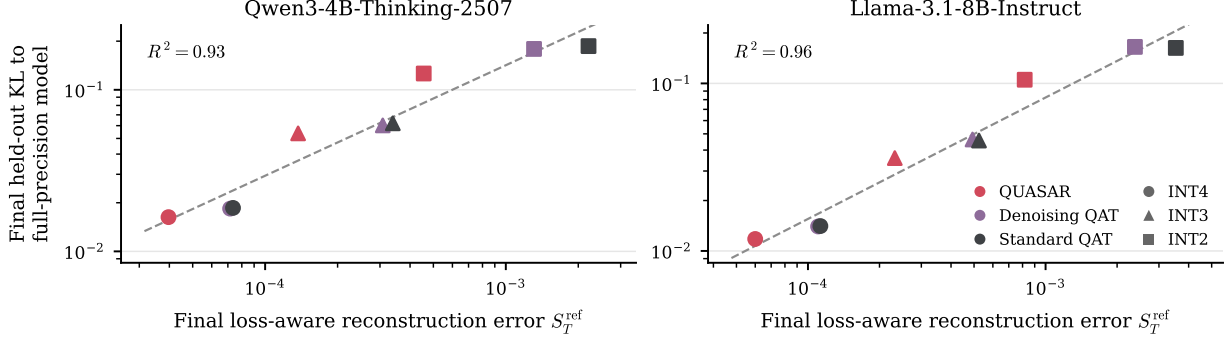


Figure 3: **Reconstruction error tracks final KL loss between the quantized model and its full-precision counterpart.** End-of-training loss-aware reconstruction error versus final held-out KL to the full-precision teacher for Standard QAT, Denoising QAT, and QUASAR at INT4, INT3, and INT2.

Then

$$\begin{aligned}
\mathbb{E}[\mathcal{L}(r_T) - \mathcal{L}^*] &\leq \frac{2L}{\mu}(1 - \eta\mu)^T(\mathcal{L}(w_0) - \mathcal{L}^*) \\
&\quad + \frac{L\eta C}{\mu} \sum_{t=0}^{T-1} (1 - \eta\mu)^{T-1-t} \mathbb{E}\hat{S}_t(r_t) \\
&\quad + \frac{L^2\eta\sigma^2}{\mu^2}(1 - (1 - \eta\mu)^T) + \frac{C}{\mu} \mathbb{E}\hat{S}_T(r_T).
\end{aligned} \tag{14}$$

In short, the result formalizes the central intuition behind QUASAR: better reconstructions throughout training translate into a tighter guarantee for the final quantized model’s loss. Proofs, the sufficient-condition derivation for C , and the supporting SGD and curvature analyses are deferred to the appendix.

5.4 An Empirical Link Between Reconstruction Error and Final KL

The theoretical analysis motivates the expectation that lower reconstruction error should accompany better quantization-aware distillation. Across Standard QAT, Denoising QAT, and QUASAR, both model families, and all tested bit widths, the reconstruction error tracks held-out KL between the quantized model and its full-precision counterpart; QUASAR has the lowest value of both in every setting (Figure 3). Because the theorem analyzes SGD whereas the main experiments use AdamW, we also repeat the INT2 comparison under plain SGD; the ordering is unchanged at every stable learning rate (Figure 10, Appendix A.2).

6 Experiments

In this section, we evaluate the effectiveness of QUASAR and compare against strong QAT and PTQ baselines. Our experiments aim to answer three research questions:

- Q1.** How well does QUASAR heal a quantized model by distilling from its full-precision counterpart, compared with existing QAT and PTQ methods?
- Q2.** How well can a quantized model learn new tasks and capabilities with QUASAR, compared with other QAT methods and with full-precision fine-tuning followed by PTQ?

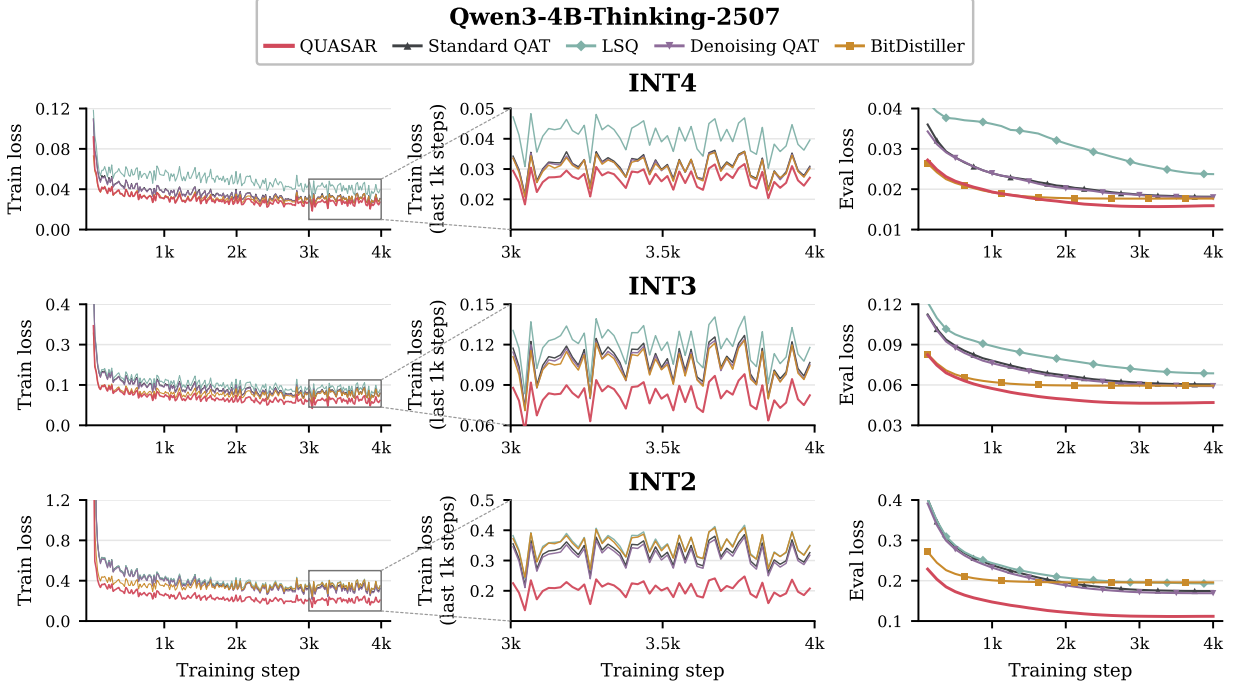


Figure 4: Quantization-aware distillation of Qwen3-4B-Thinking-2507 at INT4/INT3/INT2 on Open-PerfectBlend data using different QAT methods, with the full-precision model as teacher. From left to right: training loss over the full run, a zoomed-in view of the final 1,000 steps, and eval loss on held-out data. Training/eval loss is the forward KL between the quantized model and its full-precision counterpart.

Q3. Is QUASAR effective across quantization formats, including INT4, INT3, INT2, and NVFP4?

These questions address three practical requirements of low-bit deployment: preserving the quality of pretrained models, learning new capabilities without a lossy post-training conversion, and supporting the numerical formats native to deployment hardware. For **Q1**, quantization-aware distillation (QAD) aims to heal an already trained model in low precision, recovering as much of its full-precision quality as possible. This is especially important at two and three bits, where inference-friendly PTQ methods often degrade model quality substantially. For **Q2**, QAT enables a model to learn new tasks directly in the low-bit weights used at inference, producing a checkpoint that can be deployed without an additional quantization step. By contrast, full-precision fine-tuning followed by PTQ may weaken the newly acquired capabilities during the final, lossy conversion. For **Q3**, support for multiple numerical formats is necessary because different hardware platforms and serving scenarios impose different quantization requirements. We therefore evaluate whether QUASAR remains effective across INT4, INT3, INT2, and the NVFP4 format.

6.1 Experimental Setup

Common setup. Unless stated otherwise, we use asymmetric weight-only INT4, INT3, or INT2 quantization with group size 128, with one scale and zero-point per weight group. We quantize all linear projections in every transformer block, while keeping activations, embeddings, the language modeling head, and norms in BF16. For QAT, all weights are trained; for QAD, non-quantized modules are frozen to prevent student drift. Within each setting, all QAT methods use AdamW with the same data, objective, schedule, and token budget (4,096 steps for integer experiments).

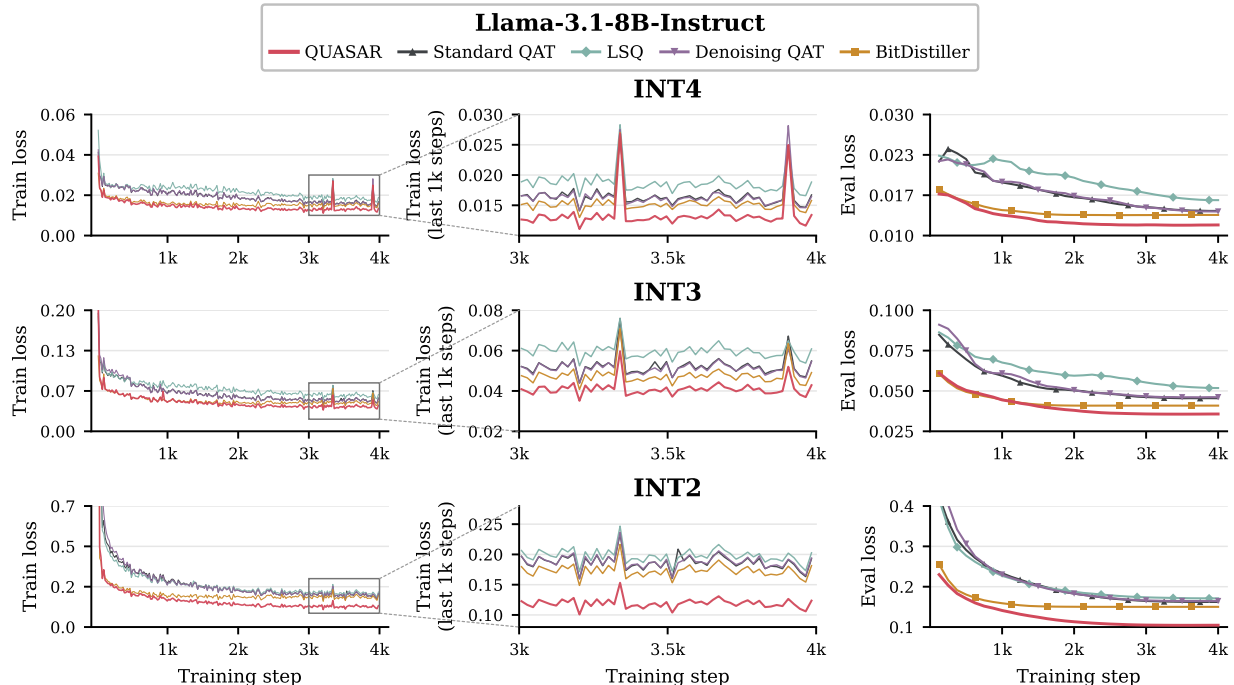


Figure 5: Quantization-aware distillation of Llama-3.1-8B-Instruct at INT4/INT3/INT2 on Open-PerfectBlend data using different QAT methods, with the full-precision model as teacher. From left to right: training loss over the full run, a zoomed-in view of the final 1,000 steps, and eval loss on held-out data. Training/eval loss is the forward KL between the quantized model and its full-precision counterpart.

We use a single learning rate per model and bit width across methods: $2/3/5 \times 10^{-5}$ for Qwen3-4B and $1/2/3 \times 10^{-5}$ for Llama-3.1-8B at INT4/INT3/INT2, with global batch sizes 32 and 128, respectively. Adaptation uses 5×10^{-5} with batch size 8, while NVFP4 QAD uses 1×10^{-6} with batch size 32. These batch sizes correspond to roughly 60k supervised tokens per step. For each model and bit width, we swept learning rates and selected the largest value stable across all methods. Higher rates tended to destabilize LSQ, while lower rates under-trained all methods; Llama required slightly lower rates than Qwen. QUASAR achieved the best performance across all learning rates evaluated. QUASAR uses AdamW’s second moment as the saliency h and searches clipping-range factors from 0.30 to 1.00 in increments of 0.05.

Healing setup. The healing experiments perform QAD on two models: Qwen3-4B-Thinking-2507 [58] and Llama-3.1-8B-Instruct [20]. The full-precision model acts as the teacher for distillation and its logits are used to train the quantized counterpart. We use the prompts from the Open-PerfectBlend dataset [30] to generate the teacher responses for distillation. We freeze the non-quantized modules and train the quantized model for 4096 steps (enough for all methods to converge) at 4096 sequence length and around 246M tokens using forward KL loss.

Adaptation setup. The adaptation experiments use Qwen3-4B-Base. Its pretraining provides enough mathematical ability to make the setting informative, while the base checkpoint still leaves a clear capability gap: it follows no instructions, so supervised fine-tuning has to teach both instruction following and long-form mathematical reasoning. We train all weights with the cross-entropy loss on OpenMathReasoning [45], at sequence length 18,432 and learning rate 5×10^{-5} .

NVFP4 setup. The NVFP4 experiments perform QAD for Qwen3-8B and Qwen3.5-9B on Open-PerfectBlend, with the same recipe as the healing experiments. Training uses simulated

Table 1: Evaluation of QAD and PTQ methods for Qwen3-4B-Thinking-2507 at INT4, INT3, and INT2 precision. PTQ directly quantizes the full-precision model, whereas QAT trains the quantized model with the full-precision model as its teacher. We report KL divergence and top-1 agreement with the full-precision model on held-out data, along with accuracy on eight downstream benchmarks.

Method	Fidelity		Downstream task accuracy (%) $\uparrow$								Avg.
	KL $\downarrow$	Top-1 $\uparrow$	GSM8K	MMLU	ARC-C	ARC-E	Hella.	WinoG.	TQA	IFEval	
Teacher (FP16)	–	–	87.0	68.7	53.1	76.7	65.7	66.1	57.5	54.3	66.1
<i>2-bit (INT2)</i>											
RTN	11.166	0.5	0.0	25.4	25.6	25.9	26.0	52.0	46.8	8.3	26.3
GPTQ	1.140	67.7	0.2	23.4	25.1	30.0	30.1	49.3	52.9	8.9	27.5
AWQ	3.016	37.0	0.0	23.4	23.2	31.6	29.1	51.1	52.8	9.6	27.6
Standard QAT	0.186	89.9	47.4	34.5	31.2	44.8	46.9	54.5	51.1	33.5	43.0
LSQ	0.205	89.2	42.6	34.6	30.6	44.9	47.6	55.2	47.2	35.1	42.2
Denoising QAT	0.179	90.1	45.6	36.0	30.1	47.9	46.7	56.1	49.9	34.9	43.4
BitDistiller	0.209	89.1	49.0	43.2	38.0	60.1	52.6	57.9	52.6	37.7	48.9
QUASAR (ours)	0.126	92.0	68.8	48.9	38.8	55.9	56.4	59.2	50.5	47.5	53.2
<i>3-bit (INT3)</i>											
RTN	0.597	78.8	20.5	53.6	39.4	56.1	54.1	58.1	51.4	18.9	44.0
GPTQ	0.130	91.5	77.3	61.0	48.0	<u>75.5</u>	61.0	61.8	54.0	49.2	61.0
AWQ	0.240	88.1	73.2	60.1	46.4	72.3	60.7	62.7	49.2	41.8	58.3
Standard QAT	0.062	94.2	78.5	62.0	46.8	74.1	61.9	64.4	<u>56.1</u>	53.2	62.1
LSQ	0.070	93.8	83.0	62.3	49.4	75.8	62.7	65.6	<u>55.4</u>	52.9	63.4
Denoising QAT	0.060	94.3	80.4	62.7	46.2	73.5	61.6	64.2	<u>56.1</u>	51.0	62.0
BitDistiller	0.065	94.2	81.0	63.7	47.0	69.0	63.0	63.5	56.3	53.4	62.1
QUASAR (ours)	0.054	94.9	<u>81.7</u>	64.5	<u>48.3</u>	72.9	63.2	<u>64.7</u>	55.5	50.6	<u>62.7</u>
<i>4-bit (INT4)</i>											
RTN	0.108	92.2	80.2	66.1	48.8	69.7	63.5	65.4	58.4	54.2	63.3
GPTQ	0.027	96.2	84.9	67.1	51.4	<u>75.0</u>	64.7	65.4	56.5	51.4	64.6
AWQ	0.050	94.7	84.6	67.6	<u>51.5</u>	74.9	65.2	66.6	56.7	<u>56.6</u>	<u>65.5</u>
Standard QAT	0.019	<u>96.9</u>	84.9	67.0	49.7	73.5	<u>65.1</u>	65.2	56.7	54.3	64.6
LSQ	0.024	96.4	<u>86.0</u>	66.8	50.6	<u>75.0</u>	65.2	65.6	57.1	56.4	65.3
Denoising QAT	0.018	<u>96.9</u>	85.0	67.1	49.2	72.9	64.7	65.7	57.3	56.9	64.9
BitDistiller	<u>0.018</u>	<u>96.9</u>	84.8	67.7	52.6	76.0	64.8	<u>65.9</u>	57.3	54.3	65.4
QUASAR (ours)	0.016	97.1	86.4	68.0	50.6	76.0	<u>65.1</u>	65.2	<u>57.7</u>	55.8	65.6

quantization. For downstream evaluation we materialize real NVFP4 checkpoints and run them through the native NVFP4 (W4A4) path of the vLLM [29] engine, so the reported quality is the quality of the deployed artifact rather than of a simulation.

QAT and PTQ baselines. We compare QUASAR with several competitive QAT and PTQ baselines. For QAT, we include Standard QAT (which determines the scale factors and zero-points based on each group’s minimum and maximum values), LSQ [14] (which jointly learns the quantization scale factors), Denoising QAT [59] (which fits the dequantization parameters with a ridge regression objective), and BitDistiller [11] (which combines tailored asymmetric quantization and clipping with self-distillation). For NVFP4, we use Standard QAT as the baseline; this matches the NVFP4 QAD recipe [47]. For PTQ, we include RTN (round to nearest), GPTQ [16] (which uses second-order information to reduce quantization error), and AWQ [35] (which uses activation statistics to identify and protect salient weights). All QAT and PTQ methods use the same inference data format for deployment, where each weight group is represented by low-bit integer codes, one scale, and one zero-point. In healing, PTQ methods quantize the full-precision model directly. In adaptation, they quantize the model after full-precision fine-tuning, which we also report without quantization as FP-SFT.

Evaluation. For healing, we evaluate teacher–student alignment on 128 samples of held-out data using forward KL and top-1 agreement rate, and report accuracy on eight benchmarks: GSM8K [8], MMLU [22], ARC-Challenge and ARC-Easy [7], HellaSwag [61], WinoGrande [53],

Table 2: Evaluation of QAD and PTQ methods for Llama-3.1-8B-Instruct at INT4, INT3, and INT2 precision. PTQ directly quantizes the full-precision model, whereas QAT trains the quantized model with the full-precision model as its teacher. We report KL divergence and top-1 agreement with the full-precision model on held-out data, along with accuracy on eight downstream benchmarks.

Method	Fidelity		Downstream task accuracy (%) $\uparrow$								
	KL $\downarrow$	Top-1 $\uparrow$	GSM8K	MMLU	ARC-C	ARC-E	Hella.	WinoG.	TQA	IFEval	Avg.
Teacher (FP16)	—	—	70.1	68.3	55.6	79.9	79.5	73.6	54.5	73.9	69.4
<i>2-bit (INT2)</i>											
RTN	10.810	0.5	0.0	25.5	26.2	26.1	26.1	51.8	47.6	11.8	26.9
GPTQ	2.380	51.8	0.0	24.9	23.3	28.9	30.0	48.0	49.0	9.1	26.7
AWQ	6.769	12.9	0.0	23.6	23.3	25.9	27.2	48.2	48.8	7.9	25.6
Standard QAT	0.163	90.1	1.8	23.4	22.9	30.9	34.8	49.3	46.8	44.4	31.8
LSQ	0.172	89.6	42.2	39.7	32.8	52.8	55.7	53.5	47.4	42.1	45.8
Denosing QAT	0.165	90.1	0.0	24.6	26.2	25.6	26.9	50.0	49.1	40.9	30.4
BitDistiller	0.151	90.2	53.1	49.3	43.5	69.9	66.6	66.3	47.7	51.8	56.0
QUASAR (ours)	0.105	92.0	66.4	52.5	43.8	73.1	68.1	66.8	49.5	56.0	59.5
<i>3-bit (INT3)</i>											
RTN	0.276	85.6	19.0	44.8	40.2	64.6	69.5	66.9	47.0	52.9	50.6
GPTQ	0.076	92.8	61.4	59.2	49.1	71.6	75.7	71.0	53.1	69.1	63.8
AWQ	0.197	88.0	23.9	53.2	43.0	67.6	72.2	67.6	42.1	58.6	53.5
Standard QAT	0.046	94.7	70.7	59.7	47.4	72.8	75.2	72.9	52.1	69.9	65.1
LSQ	0.052	94.2	76.8	62.3	51.8	76.9	75.8	71.6	50.9	67.5	66.7
Denosing QAT	0.046	94.7	68.8	59.8	48.0	74.5	75.0	<u>72.2</u>	51.0	<u>70.2</u>	64.9
BitDistiller	<u>0.041</u>	<u>94.8</u>	<u>74.2</u>	61.6	<u>53.1</u>	<u>79.0</u>	<u>76.4</u>	<u>71.9</u>	52.2	68.6	67.1
QUASAR (ours)	0.036	95.2	<u>74.1</u>	63.3	54.9	79.1	77.1	71.0	51.6	71.5	67.9
<i>4-bit (INT4)</i>											
RTN	0.044	94.4	70.7	64.1	53.3	76.9	78.0	73.9	50.7	72.1	67.5
GPTQ	0.015	96.8	69.0	67.3	52.0	77.3	<u>78.7</u>	73.2	55.1	<u>73.6</u>	68.3
AWQ	0.032	95.2	65.6	66.6	53.7	78.3	<u>78.7</u>	72.4	53.7	73.4	67.8
Standard QAT	0.014	<u>97.0</u>	76.3	66.3	54.4	77.7	78.4	73.8	53.9	73.0	69.2
LSQ	0.016	96.8	73.2	66.5	54.6	80.1	<u>78.7</u>	<u>74.1</u>	53.2	74.1	<u>69.3</u>
Denosing QAT	0.014	<u>97.0</u>	77.0	66.2	55.7	77.8	78.6	73.9	53.6	72.1	<u>69.3</u>
BitDistiller	<u>0.013</u>	<u>97.0</u>	70.4	67.0	55.1	<u>79.7</u>	78.8	73.3	53.8	74.1	69.0
QUASAR (ours)	0.012	97.3	74.6	66.5	<u>55.5</u>	78.9	78.8	74.2	<u>54.2</u>	72.5	69.4

TruthfulQA [36], and IFEval [62]. KL closely tracks task accuracy (Appendix C, Figure 22). At two bits, we additionally compare responses on 128 held-out chat prompts per model using Llama-3.3-70B-Instruct as a judge. For adaptation, we report perplexity on held-out OpenMath-Reasoning data and avg@32 accuracy at temperature 0.6 on MATH-500 [23, 34], GSM8K, AIME 2024/2025 [43], and HMMT 2025 [24]. For NVFP4, we use the healing metrics and additionally report GPQA-Diamond [51].

6.2 Results on Healing with Quantization-aware Distillation

Figures 4 and 5 show the healing results on QAD on Qwen3-4B-Thinking-2507 and Llama-3.1-8B-Instruct, respectively. The results answer **Q1**: QUASAR reaches the lowest training loss and held-out KL loss in all six settings, and every baseline plateaus above it. This matches the prediction of Section 5: lower reconstruction error should translate into a lower loss floor. At INT2, QUASAR reduces KL by about 30%, raises top-1 agreement by about 2 points, and improves average task accuracy by 3.5–4.3 points over the strongest QAT baseline (Tables 1 and 2). Below 4 bits, the best PTQ method trails QUASAR by 2–23 $\times$ in held-out KL divergence. Additional fidelity metrics and top-1 agreement during training are in Appendix C.

The gap is largest on mathematical reasoning. On GSM8K at INT2, same-format PTQ scores 0.0–0.2. QUASAR scores 68.8 on Qwen and 66.4 on Llama, while the strongest competing QAT method reaches only 49.0 and 53.1. On Llama, QUASAR is only 3.7 points below BF16.

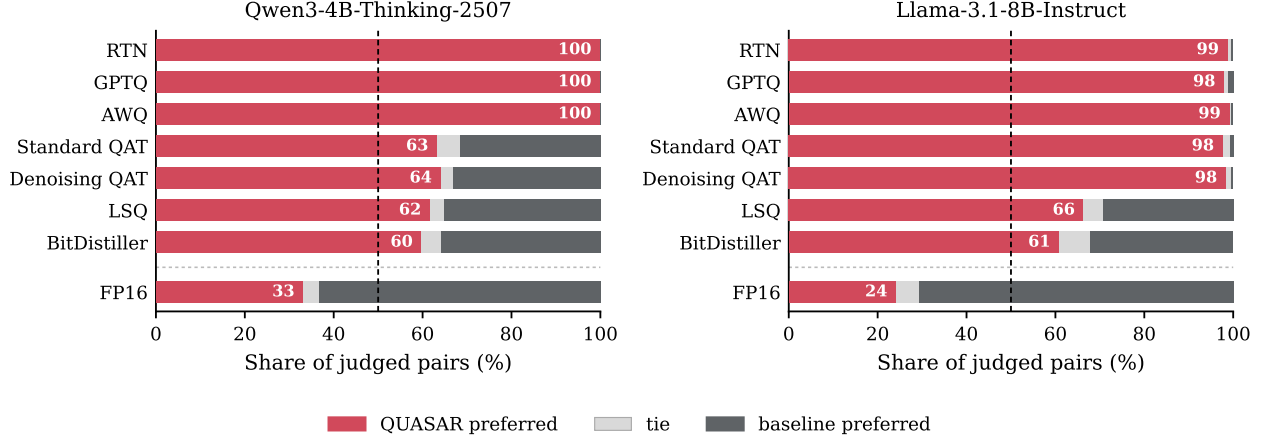


Figure 6: LLM-as-a-judge preferences for QUASAR versus the QAD and PTQ checkpoints of Qwen3-4B-Thinking-2507 and Llama-3.1-8B-Instruct at INT2, with Llama 3.3 70B serving as the judge. Each bar shows QUASAR’s win, tie, and loss rates on responses to 128 WildChat prompts.

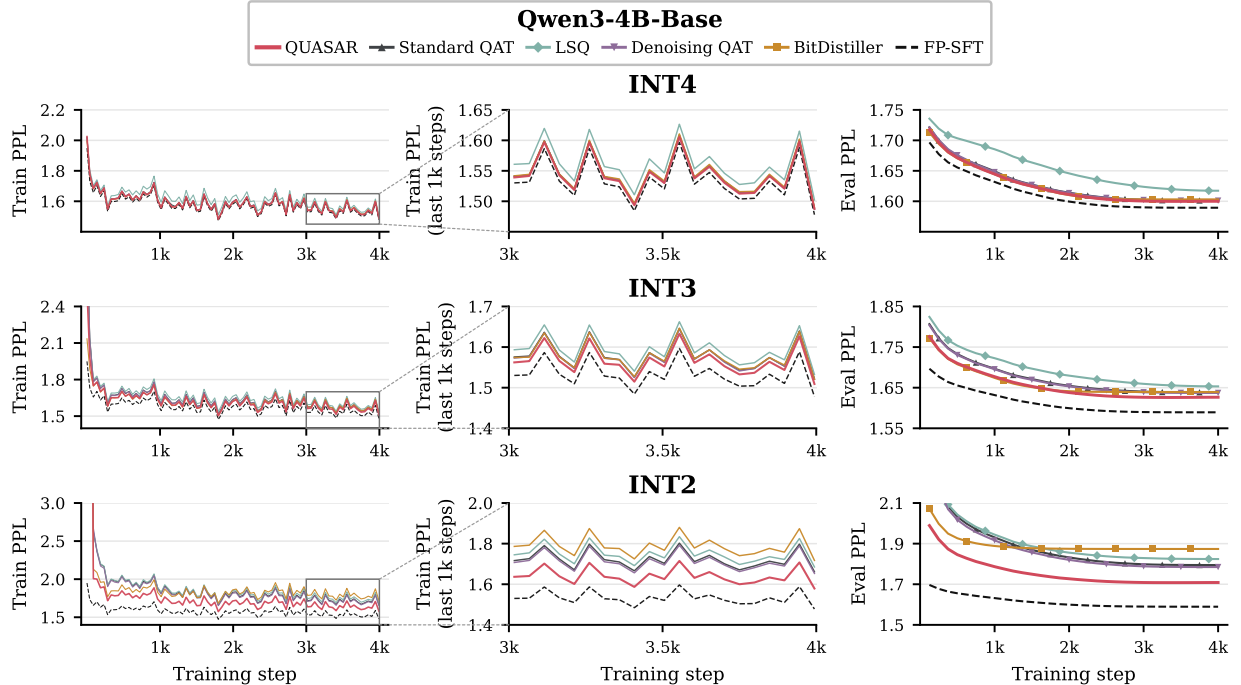


Figure 7: Quantization-aware supervised fine-tuning of Qwen3-4B-Base at INT4/INT3/INT2 on OpenMathReasoning (with chain-of-thought responses generated with DeepSeek-R1 and QwQ-32B), comparing different QAT methods and full-precision training (FP-SFT). From left to right: training perplexity, a zoomed-in view of the final 1,000 steps, and perplexity on held-out data.

LLM-as-a-judge preference at 2 bits. We use an LLM judge (Llama-3.3-70B-Instruct) to perform pairwise comparison between the responses of QUASAR checkpoints versus the PTQ and QAT checkpoints for Qwen3-4B-Thinking-2507 and Llama-3.1-8B-Instruct at INT2 on 128 WildChat prompts (Figure 6). The judge prefers QUASAR over every quantized opponent, in all 14 comparisons (Table 8) and for almost all samples over PTQ; only the full-precision teacher is preferred to QUASAR. Sample responses of each checkpoint are included in Appendix C (Figures 24 and 23).

Table 3: Evaluation of QAT and PTQ for Qwen3-4B-Base at INT4, INT3, and INT2 precision using OpenMathReasoning. QAT performs supervised fine-tuning directly at the target bit width, whereas PTQ applies quantization after full-precision fine-tuning. We report held-out perplexity and average accuracy across five math benchmarks. Base model (no SFT) and full-precision training (FP-SFT) results are included as references.

Method	Held-out fit	Downstream math accuracy (%) $\uparrow$					
	PPL $\downarrow$	MATH-500	GSM8K	AIME'24	AIME'25	HMMT'25	Avg.
Base (no SFT)	–	41.0	66.4	9.6	3.3	0.8	24.2
FP-SFT	1.59	83.8	91.3	22.9	22.5	10.0	46.1
<i>2-bit (INT2)</i>							
RTN	3.5×10^5	0.9	0.3	0.0	0.0	0.0	0.2
GPTQ	3.12	2.1	1.1	0.0	0.0	0.0	0.6
AWQ	130	1.5	1.2	0.0	0.0	0.0	0.5
Standard QAT	1.79	38.7	51.9	2.1	0.8	0.0	18.7
LSQ	1.82	36.5	49.9	0.8	0.4	0.0	17.5
Denoising QAT	1.78	39.2	52.5	0.8	0.8	0.0	18.7
BitDistiller	1.87	33.6	57.0	1.2	0.4	0.0	18.4
QUASAR (ours)	1.71	60.7	75.4	4.0	6.5	1.5	29.6
<i>3-bit (INT3)</i>							
RTN	2.24	12.8	7.6	0.0	1.7	0.0	4.4
GPTQ	1.68	71.0	85.0	11.2	11.2	4.2	36.5
AWQ	1.83	57.3	65.1	10.0	8.3	1.7	28.5
Standard QAT	1.64	75.8	87.5	15.0	16.2	4.6	39.8
LSQ	1.65	73.7	87.0	12.5	11.7	3.8	37.7
Denoising QAT	1.64	75.9	87.3	14.2	13.3	7.1	39.6
BitDistiller	1.64	75.7	87.8	16.2	15.4	5.0	40.0
QUASAR (ours)	1.62	78.5	88.6	15.9	16.2	6.0	41.1
<i>4-bit (INT4)</i>							
RTN	1.66	78.2	90.0	15.0	13.3	7.5	40.8
GPTQ	1.61	81.7	90.6	19.1	20.5	8.5	44.1
AWQ	1.63	80.3	90.5	15.8	17.5	8.3	42.5
Standard QAT	1.60	82.1	90.8	21.2	20.6	10.7	45.1
LSQ	1.62	79.0	89.1	17.9	17.1	6.7	42.0
Denoising QAT	1.60	82.1	90.4	20.0	21.4	9.9	44.8
BitDistiller	1.60	82.7	90.6	20.3	20.8	9.3	44.7
QUASAR (ours)	1.60	82.8	90.9	20.8	19.8	11.2	45.1

6.3 Results on Adaptation with QAT

Q2 asks whether a model can learn a new capability directly in the low-bit weights used at inference. The capability here is genuinely new: Qwen3-4B-Base follows no instructions, so SFT must teach it both instruction following and long-form mathematical reasoning. Figure 7 shows the adaptation curves: QUASAR reaches the lowest training and held-out perplexity at INT4, INT3, and INT2.

The gap is most visible at INT2. QUASAR stays close to FP-SFT in perplexity (1.71 vs. 1.59), while the best QAT baseline stops at 1.78. This loss gap determines whether the new capability survives: QUASAR reaches 29.6 average accuracy, 10.9 points above the strongest QAT baseline. It scores 60.7 on MATH-500 and 75.4 on GSM8K, while every fine-tune-then-quantize baseline scores at most 2.1 and 1.2, respectively (Table 3). The advantage is especially pronounced on benchmarks that require long reasoning traces. FP-SFT’s median response length is 27k tokens on AIME’24 and 20k on HMMT’25, so success requires preserving the adapted behavior over tens of thousands of generated tokens. At INT2, QUASAR is the only quantized model to score above zero on HMMT’25, showing that the 2-bit model can sustain useful reasoning deep into long generations rather than losing the capability as the trace unfolds. QUASAR is also the only INT2 arm that beats the un-finetuned base checkpoint (Appendix D, Figure 25). Appendix D examines the INT2 checkpoints further, including sampling dispersion and divergence from the teacher along reasoning traces.

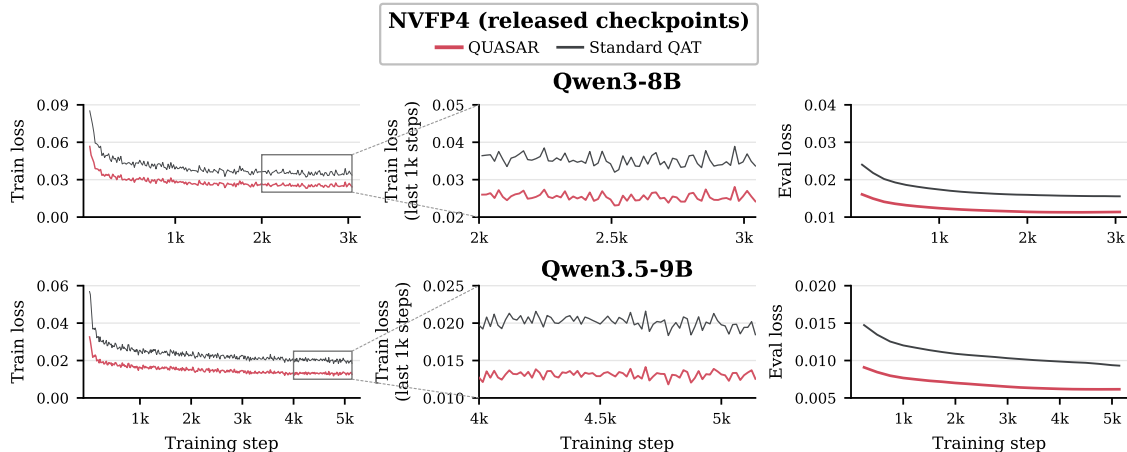


Figure 8: Training and evaluation loss of QAD for Qwen3-8B (top) and Qwen3.5-9B (bottom) in the NVFP4 format on Open-PerfectBlend data. From left to right: training loss over the full run, a zoomed-in view of the final 1,000 steps, and eval loss on held-out data. Training/eval loss is the forward KL between the quantized model and its full-precision counterpart.

6.4 Results on Quantization-aware Distillation with NVFP4

Q3 examines whether QUASAR generalizes to other numerical formats. We focus on NVFP4, a production 4-bit format for Blackwell GPUs whose quantization grid and scaling constraints differ from those of integer quantization (Section 4.2).

We perform QAD for Qwen3-8B and Qwen3.5-9B using the full-precision model as teacher and train the quantized model with simulated NVFP4 quantization on Open-PerfectBlend. We then evaluate the final real quantized NVFP4 checkpoints on downstream tasks using vLLM [29]. Our baseline is Standard QAT with a full-precision teacher [47]. Across both models, QUASAR reduces held-out KL by approximately 30% and improves average downstream accuracy by 0.5–2.0 points (Figure 8; Table 4).

Table 4: Evaluation results of different QAD methods for Qwen3-8B and Qwen3.5-9B in NVFP4. KL divergence on held-out data and accuracy on nine downstream benchmarks are reported. Evaluated in vLLM using the real NVFP4 quantized checkpoints.

Method	Fidelity		Downstream task accuracy (%) $\uparrow$								
	KL $\downarrow$	Avg.	GSM8K	MMLU	ARC-C	ARC-E	Hella.	WinoG.	TQA	IFEval	GPQA-D
<i>Qwen3-8B</i>											
BF16	—	68.6	86.7	72.9	56.7	80.9	75.0	67.7	54.4	84.8	38.4
Standard QAT	0.016	67.1	83.9	70.8	55.4	80.3	73.6	67.1	53.9	80.6	37.9
QUASAR (ours)	0.011	67.6	87.0	71.1	55.7	80.4	73.6	66.8	54.2	81.2	38.4
<i>Qwen3.5-9B</i>											
BF16	—	71.2	74.4	78.6	55.7	74.2	78.1	72.8	53.7	74.4	79.1
Standard QAT	0.009	69.4	73.3	76.7	56.3	75.6	76.7	70.8	52.8	66.0	76.1
QUASAR (ours)	0.006	70.2	73.7	77.0	56.6	75.6	77.2	70.9	53.9	68.5	78.1

6.5 Ablations and Training Overhead

Table 5 and Figure 9 ablate the two main components of QUASAR: the scale search used for code assignment and the weighted least-squares fit used to estimate the dequantization parameters. We evaluate variants that modify or remove either the scale search or the saliency weighting.

Table 5: Ablation study of QUASAR components using INT2 QAD for Qwen3-4B-Thinking-2507. Each variant modifies either scale search or saliency weighting. We report the final KL divergence between the quantized model and the full-precision counterpart on held-out data; Δ denotes the change in KL relative to Standard QAT.

Variant	Scale search		Fit weighting				KL $\downarrow$	Δ
	Every step	Initialization only	No	Adam	Fisher	None		
Standard QAT			✓			✓	0.175	–
QUASAR – Adam weighting only			✓	✓			0.164	–6.1%
QUASAR – Initialization only		✓				✓	0.172	–1.6%
QUASAR – Scale search only	✓					✓	0.114	–34.7%
QUASAR – Fisher weighting	✓				✓		0.113	–35.5%
QUASAR	✓			✓			0.112	–35.9%

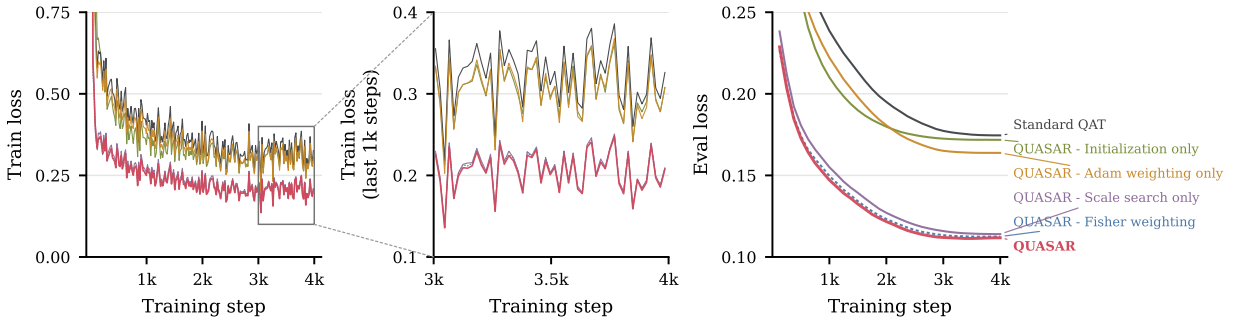


Figure 9: Component ablations of QUASAR: quantization-aware distillation of Qwen3-4B-Thinking-2507 at INT2, one curve per variant in Table 5. The loss is the forward KL between the quantized model and its full-precision counterpart. From left to right: training loss over the full run, a zoomed view of the final 1,000 steps, and the same loss on held-out data.

Two findings stand out. First, reconstruction must remain part of the training loop. Applying QUASAR only at initialization and then continuing with Standard QAT gives up nearly all of the improvement, indicating that the reconstruction needs to be updated as the weights evolve. Second, using Adam’s second moment as the saliency signal performs on par with maintaining a separate Fisher estimate, at lower computational cost.

Across both models, the median selected range factor falls from 0.95 at INT4 to 0.60 at INT2 (Appendix B, Figure 14): with fewer codes, keeping the extreme weights starves the bulk of the distribution, so the search trades them away more aggressively.

Table 6: Wall-clock time of one training step, per component and in seconds: quantization-aware distillation of Qwen3-4B-Thinking-2507 at INT3 on 8x H100 GPUs. QUASAR differs from Standard QAT only in the weight reconstruction row.

Component	Standard QAT	QUASAR
Teacher forward	0.373	0.373
Student compute	0.366	0.366
Weight reconstruction	0.294	0.335
Loss computation	0.065	0.065
Backward	1.805	1.805
Optimizer update	0.021	0.022
Other + synchronization	0.006	0.006
Total step	2.930	2.972
% vs. Standard QAT	–	+1.4%

Training overhead. Table 6 breaks down the wall-clock time for one INT3 optimizer step. QUASAR adds only 1.4% step time on a matched comparison.

7 Conclusion

We introduced QUASAR, a QAT method that mitigates the mismatch between the reconstructed weights used in the forward pass and the latent full-precision weights updated during training. QUASAR minimizes loss-aware reconstruction error by searching over quantization scales and fitting dequantization parameters via saliency-weighted least squares, using statistics derived from the optimizer. Our theoretical analysis decomposes the QAT convergence bound into three terms: initialization, minibatch noise, and loss-aware reconstruction error. Only the last depends on the reconstruction map, and it is exactly the objective QUASAR minimizes at each step. Reconstruction is therefore a direct lever on the training trajectory and, under an additional PL condition, on the loss of the final quantized model. Across two model families, multiple integer bit widths, and NVFP4, QUASAR consistently achieves the lowest training and held-out losses. These improvements translate to downstream-task benchmarks, with gains being largest at lower precision. Moreover, QUASAR matches the training speed of Standard QAT and introduces no inference overhead, while substantially improving the quality of the resulting quantized checkpoints.

References

- [1] Ahmad Ajalloeian and Sebastian U. Stich. On the convergence of SGD with biased gradients. *arXiv preprint arXiv:2008.00051*, 2020.
- [2] Eduardo Alvarez, Omri Almog, Eric Chung, Simon Layton, Dusan Stosic, Ronny Krashinsky, and Kyle Aubrey. Introducing NVFP4 for efficient and accurate low-precision inference. NVIDIA Technical Blog, June 2025. URL <https://developer.nvidia.com/blog/introducing-nvfp4-for-efficient-and-accurate-low-precision-inference/>.
- [3] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.
- [4] Léon Bottou, Frank E. Curtis, and Jorge Nocedal. Optimization methods for large-scale machine learning. *SIAM Review*, 60(2):223–311, 2018.
- [5] Jerry Chee, Yaohui Cai, Volodymyr Kuleshov, and Christopher M De Sa. Quip: 2-bit quantization of large language models with guarantees. *Advances in neural information processing systems*, 36:4396–4429, 2023.
- [6] Mengzhao Chen, Chaoyi Zhang, Jing Liu, Yutao Zeng, Zeyue Xue, Zhiheng Liu, Yunshui Li, Jin Ma, Jie Huang, Xun Zhou, and Ping Luo. Scaling law for quantization-aware training. *arXiv preprint arXiv:2505.14302*, 2025. URL <https://arxiv.org/abs/2505.14302>.
- [7] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try ARC, the AI2 reasoning challenge, 2018. URL <https://arxiv.org/abs/1803.05457>.
- [8] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and

- John Schulman. Training verifiers to solve math word problems, 2021. URL <https://arxiv.org/abs/2110.14168>.
- [9] Radosvet Desislavov, Fernando Martínez-Plumed, and José Hernández-Orallo. Trends in ai inference energy consumption: Beyond the performance-vs-parameter laws of deep learning. *Sustainable Computing: Informatics and Systems*, 38:100857, 2023.
 - [10] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems*, 2023.
 - [11] DaYou Du, Yijia Zhang, Shijie Cao, Jiaqi Guo, Ting Cao, Xiaowen Chu, and Ningyi Xu. BitDistiller: Unleashing the potential of sub-4-bit LLMs via self-distillation. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 102–116, 2024. URL <https://aclanthology.org/2024.acl-long.7/>.
 - [12] Abhinav Dutta, Sanjeev Krishnan, Nipun Kwatra, and Ramachandran Ramjee. Accuracy is not all you need. *Advances in Neural Information Processing Systems*, 37:124347–124390, 2024.
 - [13] Vage Egiazarian, Andrei Panferov, Denis Kuznedelev, Elias Frantar, Artem Babenko, and Dan Alistarh. Extreme compression of large language models via additive quantization. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.
 - [14] Steven K. Esser, Jeffrey L. McKinstry, Deepika Bablani, Rathinakumar Appuswamy, and Dharmendra S. Modha. Learned step size quantization. In *International Conference on Learning Representations*, 2020.
 - [15] Elias Frantar and Dan Alistarh. Optimal brain compression: A framework for accurate post-training quantization and pruning. *Advances in Neural Information Processing Systems*, 35: 4475–4488, 2022.
 - [16] Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. OPTQ: Accurate post-training quantization for generative pre-trained transformers. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=tcbBPnfwxS>.
 - [17] Saeed Ghadimi and Guanghui Lan. Stochastic first- and zeroth-order methods for nonconvex stochastic programming. *SIAM Journal on Optimization*, 23(4):2341–2368, 2013.
 - [18] Amir Gholami, Sehoon Kim, Zhen Dong, Zhewei Yao, Michael W Mahoney, and Kurt Keutzer. A survey of quantization methods for efficient neural network inference. *arXiv preprint arXiv:2103.13630*, 2021.
 - [19] Ruihao Gong, Xianglong Liu, Shenghu Jiang, Tianxiang Li, Peng Hu, Jiazhen Lin, Fengwei Yu, and Junjie Yan. Differentiable soft quantization: Bridging full-precision and low-bit neural networks. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.
 - [20] Aaron Grattafiori et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. URL <https://arxiv.org/abs/2407.21783>.

- [21] Babak Hassibi and David G. Stork. Second order derivatives for network pruning: Optimal brain surgeon. In *Advances in Neural Information Processing Systems*, volume 5, pages 164–171, 1992. URL <https://proceedings.neurips.cc/paper/1992/hash/303ed4c69846ab36c2904d3ba8573050-Abstract.html>.
- [22] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2021.
- [23] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *NeurIPS Datasets and Benchmarks Track*, 2021.
- [24] HMMT. Harvard–MIT mathematics tournament (HMMT). <https://www.hmmt.org>.
- [25] Alec Ibarra, Bryan Shan, Daniel Nishball, et al. Vera rubin NVL72 vs. GB200 NVL72? inference TCO & architecture analysis. <https://newsletter.semianalysis.com/p/vera-rubin-nvl72-vs-gb200-nvl72-inference>, July 2026.
- [26] Hamed Karimi, Julie Nutini, and Mark Schmidt. Linear convergence of gradient and proximal-gradient methods under the Polyak–Łojasiewicz condition. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases (ECML PKDD)*, 2016.
- [27] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.
- [28] Tanishq Kumar, Zachary Ankner, Benjamin Frederick Spector, Blake Bordelon, Niklas Muenighoff, Mansheej Paul, Cengiz Pehlevan, Christopher Ré, and Aditi Raghunathan. Scaling laws for precision. In *International Conference on Learning Representations*, 2025. URL <https://arxiv.org/abs/2411.04330>.
- [29] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pages 611–626, 2023.
- [30] Maxime Labonne. Open-PerfectBlend: An open reproduction of the instruction mixture of “the perfect blend”. <https://huggingface.co/datasets/mlabonne/open-perfectblend>, 2024. Hugging Face dataset; reproduces the mixture of arXiv:2409.20370.
- [31] Dongyue Li, Zechun Liu, Kai Yi, Zhenshuo Zhang, Changsheng Zhao, Raghuraman Krishnamoorthi, Harshit Khaitan, Hongyang R. Zhang, and Steven Li. WinQ: Accelerating quantization-aware training of language models around saddle points. In *Proceedings of the 43rd International Conference on Machine Learning*, 2026.
- [32] Yuhang Li, Ruihao Gong, Xu Tan, Yang Yang, Peng Hu, Qi Zhang, Fengwei Yu, Wei Wang, and Shi Gu. BRECCQ: Pushing the limit of post-training quantization by block reconstruction. In *International Conference on Learning Representations*, 2021. URL <https://openreview.net/forum?id=POWv6hDd9XH>.
- [33] Zhen Li, Yupeng Su, Runming Yang, Congkai Xie, Zheng Wang, Zhongwei Xie, Ngai Wong, and Hongxia Yang. Quantization meets reasoning: Exploring llm low-bit quantization degradation for mathematical reasoning. *arXiv preprint arXiv:2501.03035*, 2025.

- [34] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step, 2023. URL <https://arxiv.org/abs/2305.20050>.
- [35] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. In *Proceedings of Machine Learning and Systems*, volume 6, pages 87–100, 2024. URL https://proceedings.mlsys.org/paper_files/paper/2024/hash/42a452cbafa9dd64e9ba4aa95cc1ef21-Abstract-Conference.html.
- [36] Stephanie Lin, Jacob Hilton, and Owain Evans. TruthfulQA: Measuring how models mimic human falsehoods. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, 2022.
- [37] Yujun Lin, Haotian Tang, Shang Yang, Zhekai Zhang, Guangxuan Xiao, Chuang Gan, and Song Han. QServe: W4A8KV4 quantization and system co-design for efficient LLM serving. In *Proceedings of Machine Learning and Systems*, volume 7, 2025.
- [38] Chaoyue Liu, Libin Zhu, and Mikhail Belkin. Loss landscapes and optimization in over-parameterized non-linear systems and neural networks. *Applied and Computational Harmonic Analysis*, 59:85–116, 2022.
- [39] Ruikang Liu, Yuxuan Sun, Manyi Zhang, Haoli Bai, Xianzhi Yu, Tiezheng Yu, Chun Yuan, and Lu Hou. Quantization hurts reasoning? an empirical study on quantized reasoning models. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=BM192Ps5Nv>.
- [40] Zechun Liu, Barlas Oguz, Changsheng Zhao, Ernie Chang, Pierre Stock, Yashar Mehdad, Yangyang Shi, Raghuraman Krishnamoorthi, and Vikas Chandra. LLM-QAT: Data-free quantization aware training for large language models. *arXiv preprint arXiv:2305.17888*, 2023.
- [41] Kelly Marchisio, Saurabh Dash, Hongyu Chen, Dennis Aumiller, Ahmet Üstün, Sara Hooker, and Sebastian Ruder. How does quantization affect multilingual llms? In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 15928–15947, 2024.
- [42] Federico Marcuzzi, Xuefei Ning, Roy Schwartz, and Iryna Gurevych. How quantization shapes bias in large language models. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 363–404, 2026.
- [43] Mathematical Association of America. American invitational mathematics examination (AIME). <https://maa.org/maa-invitational-competitions>.
- [44] Anmol Mekala, Anirudh Atmakuru, Yixiao Song, Marzena Karpinska, and Mohit Iyyer. Does quantization affect models’ performance on long-context tasks? In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 9422–9470, 2025. URL <https://aclanthology.org/2025.emnlp-main.479/>.
- [45] Ivan Moshkov, Darragh Hanley, Ivan Sorokin, Shubham Toshniwal, Christof Henkel, Benedikt Schifferer, Wei Du, and Igor Gitman. AIMO-2 winning solution: Building state-of-the-art mathematical reasoning models with OpenMathReasoning dataset, 2025. URL <https://arxiv.org/abs/2504.16891>.

- [46] Markus Nagel, Rana Ali Amjad, Mart Van Baalen, Christos Louizos, and Tijmen Blankevoort. Up or down? Adaptive rounding for post-training quantization. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 7197–7206. PMLR, 2020. URL <https://proceedings.mlr.press/v119/nagel20a.html>.
- [47] NVIDIA. Quantization-aware distillation for NVFP4 inference accuracy recovery. Technical report, NVIDIA, 2026. URL <https://research.nvidia.com/labs/nemotron/files/NVFP4-QAD-Report.pdf>.
- [48] NVIDIA, Felix Abecassis, Anjulie Agrusa, Dong Ahn, Jonah Alben, Stefania Alborghetti, Michael Andersch, Sivakumar Arayandi, Alexis Bjorlin, Aaron Blakeman, Evan Briones, et al. Pretraining large language models with NVFP4. *arXiv preprint arXiv:2509.25149*, 2025.
- [49] NVIDIA Corporation. Parallel Thread Execution ISA, version 9.4. https://docs.nvidia.com/cuda/developer-preview/13.4/pdf/ptx_isa_9.4.pdf, 2026. CUDA 13.4 Developer Preview.
- [50] Boris T. Polyak. Gradient methods for the minimisation of functionals. *USSR Computational Mathematics and Mathematical Physics*, 3(4):864–878, 1963.
- [51] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. GPQA: A graduate-level google-proof Q&A benchmark, 2023. URL <https://arxiv.org/abs/2311.12022>.
- [52] Bitu Darvish Rouhani, Ritchie Zhao, Ankit More, Mathew Hall, Alireza Khodamoradi, Summer Deng, Dhruv Choudhary, Marius Cornea, Eric Dellinger, Kristof Denolf, et al. Microscaling data formats for deep learning. *arXiv preprint arXiv:2310.10537*, 2023.
- [53] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. WinoGrande: An adversarial winograd schema challenge at scale. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2020.
- [54] Nikhil Sardana, Jacob Portes, Sasha Doubov, and Jonathan Frankle. Beyond Chinchilla-optimal: Accounting for inference in language model scaling laws. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 43445–43460. PMLR, 2024. URL <https://proceedings.mlr.press/v235/sardana24a.html>.
- [55] Albert Tseng, Jerry Chee, Qingyao Sun, Volodymyr Kuleshov, and Christopher De Sa. QuIP#: Even better LLM quantization with hadamard incoherence and lattice codebooks. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.
- [56] Albert Tseng, Qingyao Sun, David Hou, and Christopher De Sa. QTIP: Quantization with trellises and incoherence processing. In *Advances in Neural Information Processing Systems*, 2024.
- [57] Albert Tseng, Zhaofeng Sun, and Christopher De Sa. Model-preserving adaptive rounding. In *Proceedings of the 43rd International Conference on Machine Learning*, 2026.
- [58] An Yang et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. URL <https://arxiv.org/abs/2505.09388>.

- [59] Chengxi Ye, Grace Chu, Yanfeng Liu, Yichi Zhang, Lukasz Lew, Li Zhang, Mark Sandler, and Andrew G. Howard. Robust training of neural networks at arbitrary precision and sparsity. In *International Conference on Learning Representations*, 2026. URL <https://iclr.cc/virtual/2026/poster/10008354>.
- [60] Penghang Yin, Jiancheng Lyu, Shuai Zhang, Stanley Osher, Yingyong Qi, and Jack Xin. Understanding straight-through estimator in training activation quantized neural nets. In *International Conference on Learning Representations*, 2019.
- [61] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019.
- [62] Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models, 2023. URL <https://arxiv.org/abs/2311.07911>.

Appendix

A Proofs

This appendix contains the proofs of the results stated in Section 5.

A.1 Proofs of the main results

We use the notation $\mathbb{E}_t[\cdot] = \mathbb{E}[\cdot \mid \mathcal{H}_t]$ introduced in Section 5. Let us first verify the sufficient condition for Assumption 3 given there. If $\mathcal{L}$ is L -smooth and $h_{t,i} \geq h_{\min} > 0$, then, for $y, d \in \mathbb{R}^d$,

$$\|\nabla^2 \mathcal{L}(y)d\|^2 \leq L^2 \|d\|^2 \leq \frac{L^2}{h_{\min}} \sum_i h_{t,i} d_i^2.$$

Consequently, (9) holds with $C = L^2/h_{\min}$.

Proof of Theorem 1. Fix $t < T$ and put $d_t = r_t - w_t$. By the fundamental theorem of calculus,

$$\nabla \mathcal{L}(r_t) - \nabla \mathcal{L}(w_t) = \int_0^1 \nabla^2 \mathcal{L}(w_t + ud_t) d_t du.$$

Jensen's inequality and Assumption 3 now give

$$\begin{aligned} \|\nabla \mathcal{L}(r_t) - \nabla \mathcal{L}(w_t)\|^2 &\leq \int_0^1 \|\nabla^2 \mathcal{L}(w_t + ud_t) d_t\|^2 du \\ &\leq C \widehat{S}_t(r_t) = C \min_{r \in \mathcal{R}_t} \widehat{S}_t(r), \end{aligned}$$

where the last equality follows from the definition of r_t . This proves the first assertion.

We turn to the convergence bound. Set

$$a_t = \nabla \mathcal{L}(w_t), \quad c_t = \nabla \mathcal{L}(r_t).$$

By smoothness and the update in Assumption 2,

$$\begin{aligned} \mathcal{L}(w_{t+1}) &\leq \mathcal{L}(w_t) - \eta \langle a_t, c_t + \xi_t \rangle \\ &\quad + \frac{L\eta^2}{2} \|c_t + \xi_t\|^2. \end{aligned}$$

Conditional on $\mathcal{H}_t$, both a_t and c_t are fixed. Moreover,

$$\mathbb{E}_t \|c_t + \xi_t\|^2 = \|c_t\|^2 + \mathbb{E}_t \|\xi_t\|^2 \leq (1 + M) \|c_t\|^2 + \sigma^2.$$

It follows that

$$\mathbb{E}_t[\mathcal{L}(w_{t+1})] \leq \mathcal{L}(w_t) - \eta \langle a_t, c_t \rangle + \frac{L\eta^2}{2} \left((1 + M) \|c_t\|^2 + \sigma^2 \right).$$

Using the identity

$$2\langle a_t, c_t \rangle = \|a_t\|^2 + \|c_t\|^2 - \|c_t - a_t\|^2$$

and the first assertion, we obtain

$$\begin{aligned}\mathbb{E}_t[\mathcal{L}(w_{t+1})] &\leq \mathcal{L}(w_t) - \frac{\eta}{2} \|a_t\|^2 \\ &\quad - \frac{\eta(1 - L\eta(1 + M))}{2} \|c_t\|^2 + \frac{\eta C}{2} \widehat{S}_t(r_t) + \frac{L\eta^2}{2} \sigma^2.\end{aligned}\tag{15}$$

We take expectations and sum this inequality for $0 \leq t < T$. The terms involving the loss telescope, and therefore

$$\begin{aligned}\frac{\eta}{2} \sum_{t=0}^{T-1} \mathbb{E} \left[\|\nabla \mathcal{L}(w_t)\|^2 + (1 - L\eta(1 + M)) \|\nabla \mathcal{L}(r_t)\|^2 \right] &\leq \mathcal{L}(w_0) - \mathbb{E} \mathcal{L}(w_T) \\ &\quad + \frac{\eta C}{2} \sum_{t=0}^{T-1} \mathbb{E} \widehat{S}_t(r_t) + \frac{L\eta^2 T}{2} \sigma^2.\end{aligned}$$

Since $\mathcal{L}(w_T) \geq \mathcal{L}^*$, the desired result follows after division by $\eta T/2$. $\square$

Proof of Corollary 1. Put $q = 1 - \eta\mu$. The assumption on η ensures that $0 < q < 1$ and that $1 - L\eta(1 + M) > 0$. We may thus omit the term involving $\|c_t\|^2$ in (15). The PL inequality at w_t gives

$$\mathbb{E}[\mathcal{L}(w_{t+1}) - \mathcal{L}^*] \leq q \mathbb{E}[\mathcal{L}(w_t) - \mathcal{L}^*] + \frac{\eta C}{2} \mathbb{E} \widehat{S}_t(r_t) + \frac{L\eta^2}{2} \sigma^2.$$

Iteration yields

$$\begin{aligned}\mathbb{E}[\mathcal{L}(w_T) - \mathcal{L}^*] &\leq q^T (\mathcal{L}(w_0) - \mathcal{L}^*) \\ &\quad + \frac{\eta C}{2} \sum_{t=0}^{T-1} q^{T-1-t} \mathbb{E} \widehat{S}_t(r_t) \\ &\quad + \frac{L\eta\sigma^2}{2\mu} (1 - q^T).\end{aligned}\tag{16}$$

It remains to compare w_T and r_T . Applying the smoothness inequality at w_T with the point $w_T - L^{-1} \nabla \mathcal{L}(w_T)$, we get

$$\mathcal{L}\left(w_T - \frac{1}{L} \nabla \mathcal{L}(w_T)\right) \leq \mathcal{L}(w_T) - \frac{1}{2L} \|\nabla \mathcal{L}(w_T)\|^2.$$

The left-hand side is at least $\mathcal{L}^*$, and hence

$$\|\nabla \mathcal{L}(w_T)\|^2 \leq 2L(\mathcal{L}(w_T) - \mathcal{L}^*).$$

The argument used for the first assertion of the theorem, now applied to the terminal reconstruction, also gives

$$\|\nabla \mathcal{L}(r_T) - \nabla \mathcal{L}(w_T)\|^2 \leq C \widehat{S}_T(r_T).$$

By the PL inequality at r_T and $\|x + y\|^2 \leq 2\|x\|^2 + 2\|y\|^2$, it follows that

$$\begin{aligned}\mathcal{L}(r_T) - \mathcal{L}^* &\leq \frac{1}{2\mu} \|\nabla \mathcal{L}(r_T)\|^2 \\ &\leq \frac{1}{\mu} \|\nabla \mathcal{L}(w_T)\|^2 + \frac{1}{\mu} \|\nabla \mathcal{L}(r_T) - \nabla \mathcal{L}(w_T)\|^2 \\ &\leq \frac{2L}{\mu} (\mathcal{L}(w_T) - \mathcal{L}^*) + \frac{C}{\mu} \widehat{S}_T(r_T).\end{aligned}$$

We finally take expectations and use (16). Since $q = 1 - \eta\mu$, this is precisely (14). $\square$

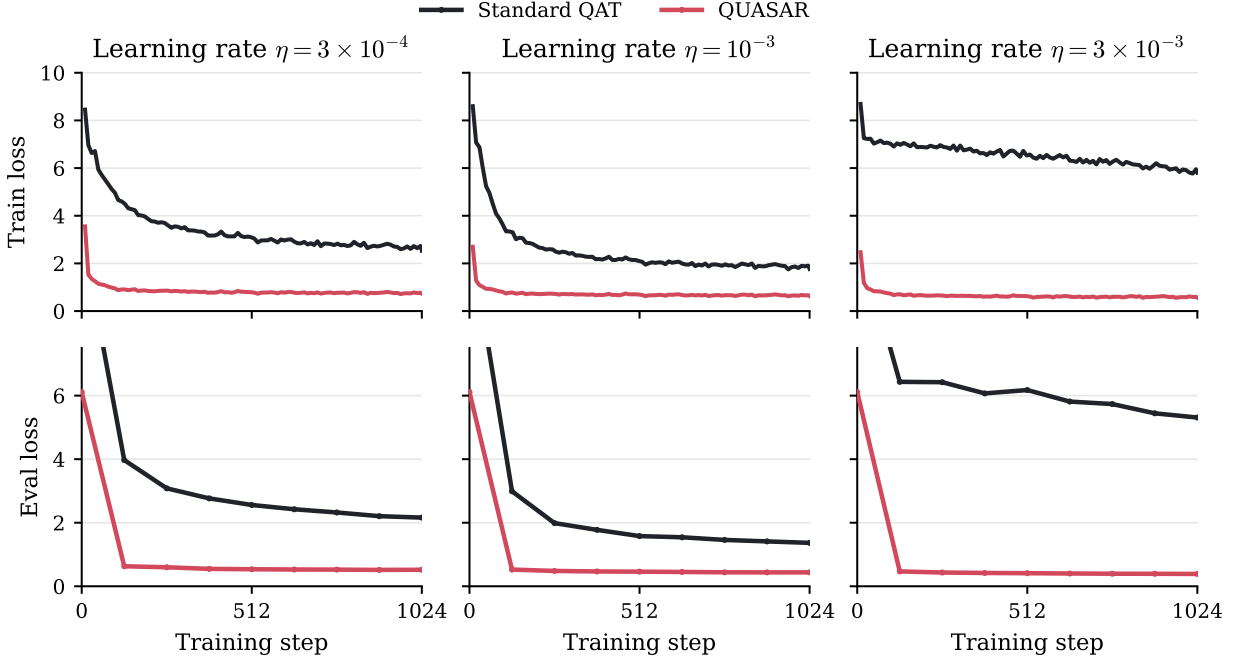


Figure 10: Standard QAT and QUASAR under plain SGD (no momentum, constant learning rate, no warmup) on the INT2 quantization-aware distillation recipe.

A.2 Validation under the SGD dynamics

Theorem 1 analyzes STE-SGD, while all main experiments train with AdamW. To check that the predicted ordering is not an optimizer artifact, we repeat the INT2 quantization-aware distillation comparison under the analyzed dynamics: plain SGD (no momentum, constant learning rate, no warmup, no gradient clipping) for 1,024 steps at global batch 32, with the recipe otherwise unchanged. QUASAR’s saliency comes from an explicitly maintained EMA of squared gradients instead of AdamW’s second moment. At every stable learning rate, QUASAR reaches a lower train and held-out loss floor than Standard QAT, matching the ordering under AdamW (Figure 10).

A.3 Choice of saliency weighting

The score $\hat{S}_t$ weights squared perturbations by the saliency h_t , which tracks the diagonal curvature. Since the linearized gradient mismatch is $\|H_t \Delta_t\|^2 = \Delta_t^\top H_t^2 \Delta_t$, a natural alternative is to weight by h_t^2 and select the reconstruction that minimizes an estimate of the mismatch itself. We compare the two weightings with everything else fixed: under AdamW at the INT2, INT3, and INT4 distillation learning rates, and under plain SGD at the three stable learning rates of Figure 10. The h_t weighting reaches a lower held-out loss in all six comparisons (Figure 11). The squared weighting is also unstable: it collapses at INT4 and at the larger SGD learning rates.

B Method Details

This appendix measures, on real weight groups, what Sections 3.1 and 4 illustrate: the per-weight reconstruction error, the clipping ranges scale search selects, and the saliency signal h . It ends by instantiating QUASAR for the NVIDIA Rubin INT3 lookup-table format, as Section 4.2 does for NVFP4.

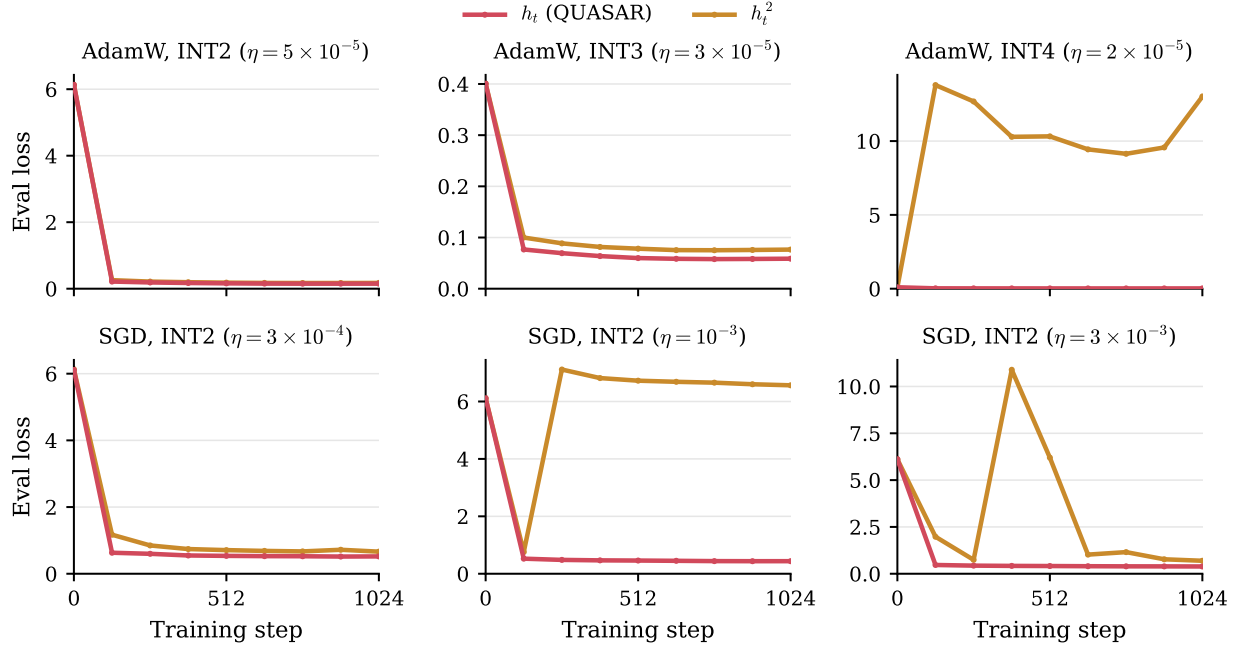


Figure 11: Held-out loss for QUASAR selecting reconstructions with saliency weights h_t versus h_t^2 , under AdamW at the INT2, INT3, and INT4 distillation learning rates (top) and under plain SGD at the three stable INT2 learning rates (bottom).



Figure 12: Per-weight loss-aware reconstruction error $|\sqrt{h}(r-w)|$ for 14 weight groups sampled from Qwen3-4B at INT3, under (a) Standard QAT's min-max reconstruction and (b) QUASAR's. Each row is one group of 128 weights, sorted by saliency h .

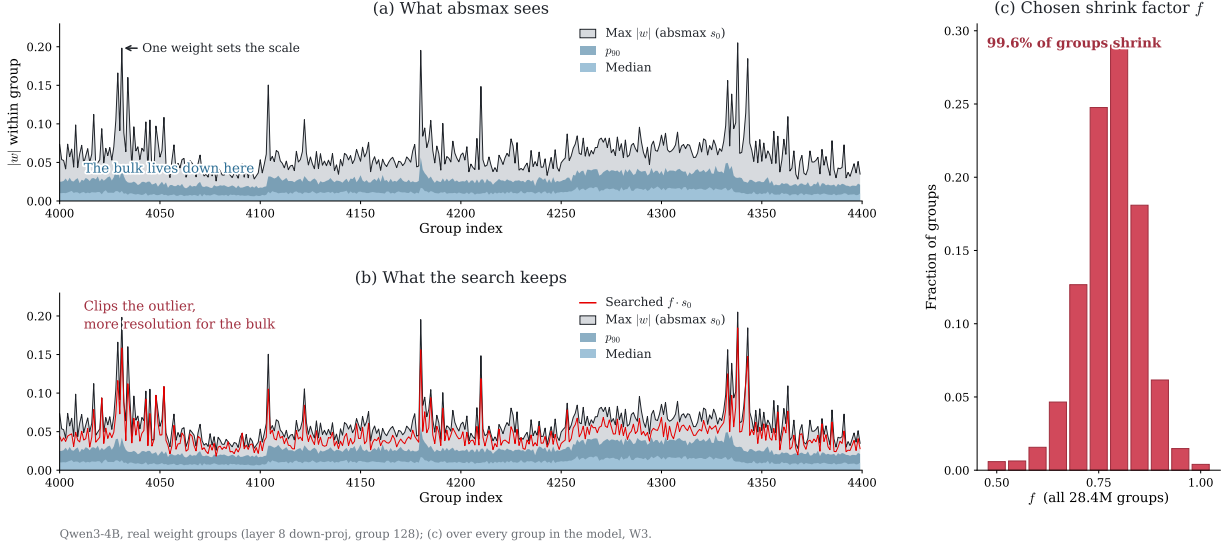


Figure 13: Clipping ranges of weight groups (Qwen3-4B, INT3). (a) The full min-max range of each group is set by one extreme weight, far above the bulk. (b) The searched range clips that weight and follows the bulk. (c) Distribution of the selected range factors f^* across all 28.4M groups; 99.6% of groups select a range narrower than min-max.

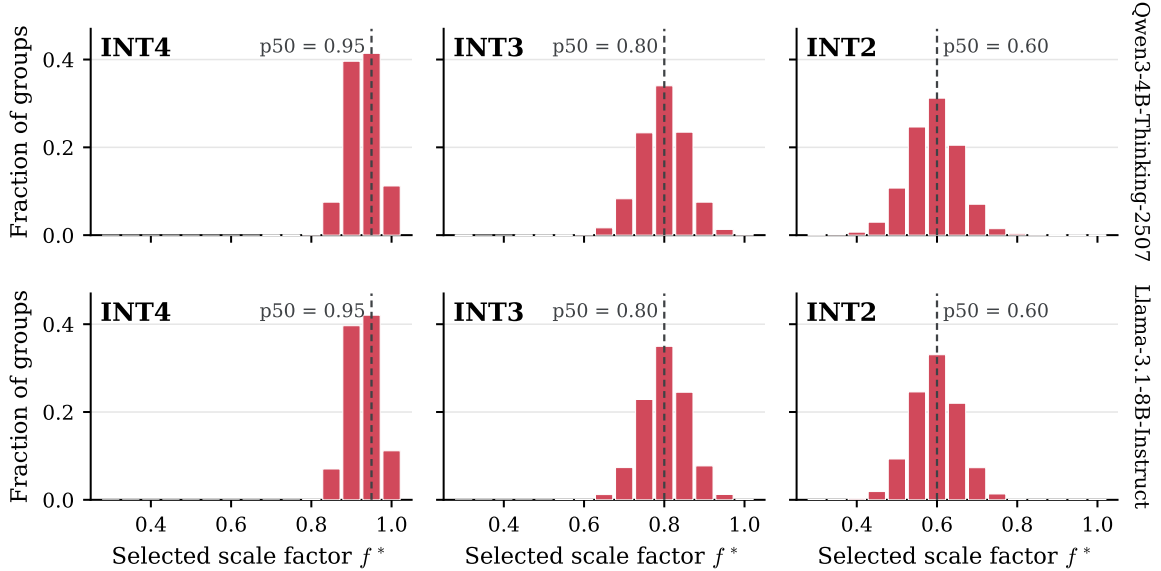


Figure 14: Distribution of the selected clipping-range factor f^* across weight groups at the end of QAD runs at INT4/INT3/INT2 for Qwen3-4B-Thinking-2507 and Llama-3.1-8B-Instruct.

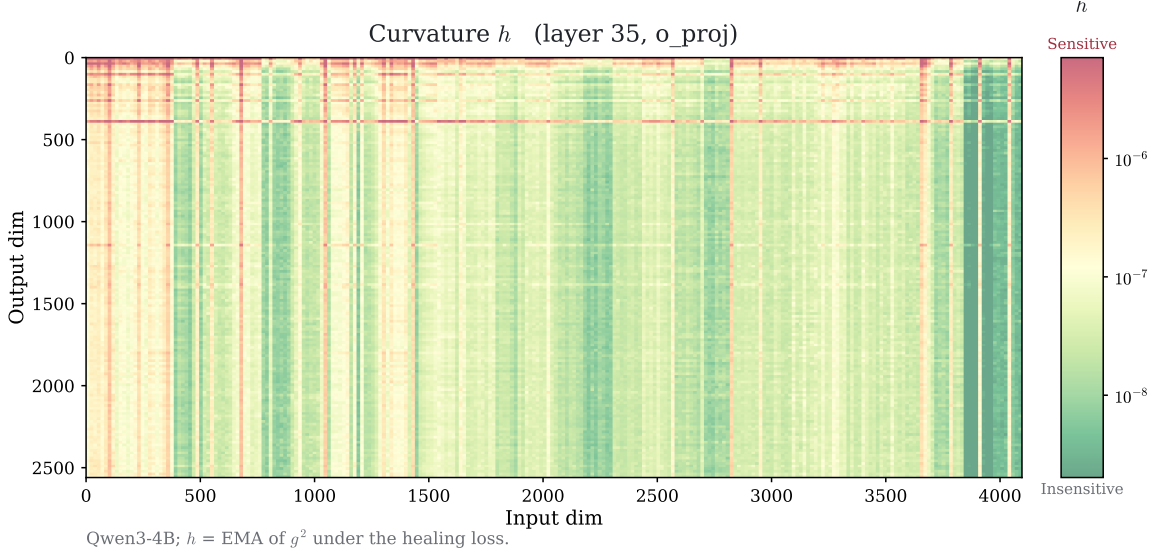


Figure 15: The saliency signal h over one layer of Qwen3-4B. QUASAR’s weight groups run along the input dimension, 128 weights per group. h varies strongly within each group, so the weighting tells QUASAR which weights to reconstruct most accurately.

Instantiation for NVIDIA Rubin INT3. Rubin INT3 replaces affine dequantization with a hardware lookup table [25, 49]. The correspondence with the main method is:

Component	Affine QUASAR	Rubin INT3 LUT-QUASAR
Stored code	Integer q_i	3-bit index $q_i \in \{0, \dots, 7\}$
Dequantization	$r_i = sq_i + z$	$r_i = C_{q_i}$
Dequantization parameters	(s, z)	$\{C_0, \dots, C_7\}$ (E4M3)
Hardware block	Group of g weights	$8 \times 64 = 512$ weights
Export	Codes, scale, and zero-point	Packed indices and codebook

For fixed indices, the optimal hardware-representable value for each used entry is its loss-aware mean rounded to E4M3:

$$\begin{aligned} \hat{S}_t(q, C) &= \sum_{i=1}^{512} h_i (C_{q_i} - w_i)^2, \\ C_k^* &= Q_{\text{E4M3}} \left(\frac{\sum_{i:q_i=k} h_i w_i}{\sum_{i:q_i=k} h_i} \right), \end{aligned} \tag{17}$$

where Q_{E4M3} applies the target hardware’s rounding.

For each candidate clipping range $f \in \mathcal{F}$, QUASAR assigns indices q_f , computes C_f^* using Equation (17), evaluates $\hat{S}_t(q_f, C_f^*)$, and retains the best reconstruction.

Each block stores 512 packed 3-bit indices and eight 8-bit E4M3 entries, for

$$\frac{512 \cdot 3 + 8 \cdot 8}{512} = 3.125 \quad \text{bits per weight.}$$

After training, curvature estimation, candidate search, and codebook fitting are discarded. The exported checkpoint is an ordinary Rubin INT3 LUT checkpoint.

C Additional Healing Results

This appendix provides additional healing results for Section 6.2: more evaluation metrics for the final checkpoints, training dynamics, and the response quality of the INT2 checkpoints, including a full example response across all methods.

Table 7: Additional evaluation metrics for QAD of Qwen3-4B-Thinking-2507 and Llama-3.1-8B-Instruct at INT4/INT3/INT2, for the same final quantized checkpoints as Tables 1 and 2: reverse KL, 99th-percentile per-token KL, and top-1 agreement on tokens where the full-precision model is uncertain (top-1 probability < 0.9). Best result per bit width in **bold**; second best underlined.

Method	Qwen3-4B-Thinking-2507			Llama-3.1-8B-Instruct		
	Rev. KL ↓	P99 KL ↓	Low-conf. Top-1 ↑	Rev. KL ↓	P99 KL ↓	Low-conf. Top-1 ↑
<i>2-bit (INT2)</i>						
RTN	33.336	19.731	0.4	19.936	16.897	0.3
GPTQ	4.357	8.494	32.0	8.100	12.461	25.4
AWQ	14.386	11.959	17.8	16.781	15.193	6.9
Standard QAT	0.361	2.459	59.9	0.264	2.094	68.3
LSQ	0.407	2.631	57.8	0.283	2.167	67.2
Denosing QAT	<u>0.346</u>	<u>2.386</u>	<u>60.5</u>	0.270	2.148	68.5
BitDistiller	0.413	2.630	57.8	<u>0.242</u>	<u>1.895</u>	68.7
QUASAR (ours)	0.201	1.702	65.9	0.153	1.380	73.4
<i>3-bit (INT3)</i>						
RTN	1.490	5.868	41.0	0.541	2.736	57.0
GPTQ	0.172	1.915	63.8	0.105	0.931	75.7
AWQ	0.338	3.253	55.4	0.325	2.067	62.2
Standard QAT	0.085	0.930	73.3	0.056	0.604	81.5
LSQ	0.098	1.020	71.5	0.064	0.675	80.0
Denosing QAT	<u>0.083</u>	<u>0.914</u>	<u>73.5</u>	0.055	0.611	81.5
BitDistiller	0.091	0.944	73.3	0.050	0.544	82.0
QUASAR (ours)	0.070	0.769	76.1	0.043	0.476	83.2
<i>4-bit (INT4)</i>						
RTN	0.152	1.568	66.2	0.056	0.487	80.4
GPTQ	0.030	0.426	81.3	0.016	0.187	88.8
AWQ	0.054	0.746	74.7	0.035	0.389	83.1
Standard QAT	0.020	0.301	84.8	0.015	0.188	89.3
LSQ	0.028	0.378	82.4	0.017	0.205	88.8
Denosing QAT	<u>0.020</u>	<u>0.295</u>	84.7	0.015	0.188	<u>89.5</u>
BitDistiller	0.020	0.299	84.9	0.014	0.176	89.5
QUASAR (ours)	0.018	0.265	85.6	0.013	0.154	90.3

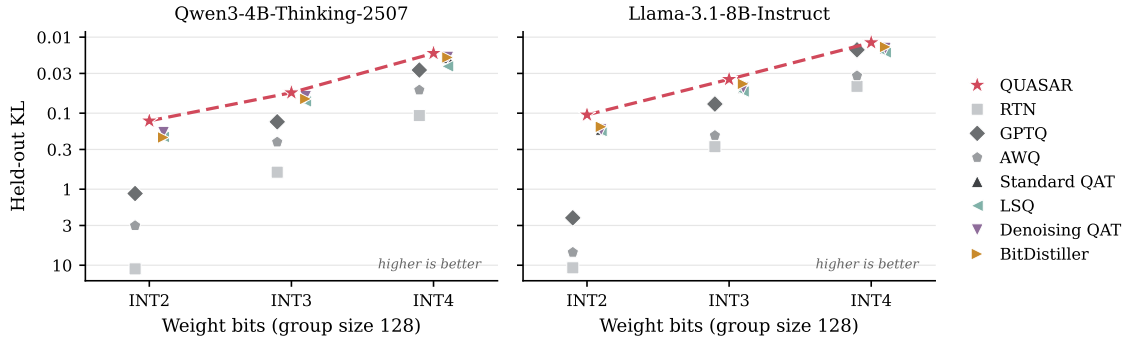


Figure 16: Forward KL between the quantized model and its full-precision counterpart on held-out data, versus bit width, for QAD of Qwen3-4B-Thinking-2507 (left) and Llama-3.1-8B-Instruct (right). Higher is better.

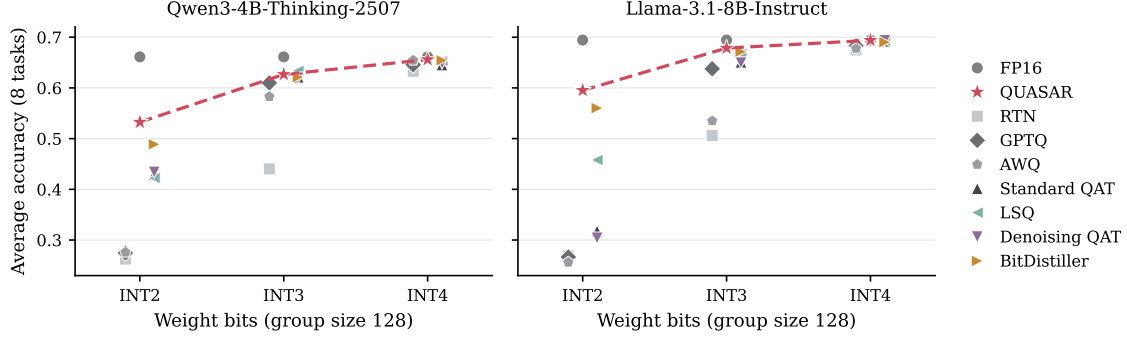


Figure 17: Average accuracy over the eight benchmarks of Table 1 and 2 versus bit width, for QAD of Qwen3-4B-Thinking-2507 (left) and Llama-3.1-8B-Instruct (right). Gray circles mark the full-precision models.

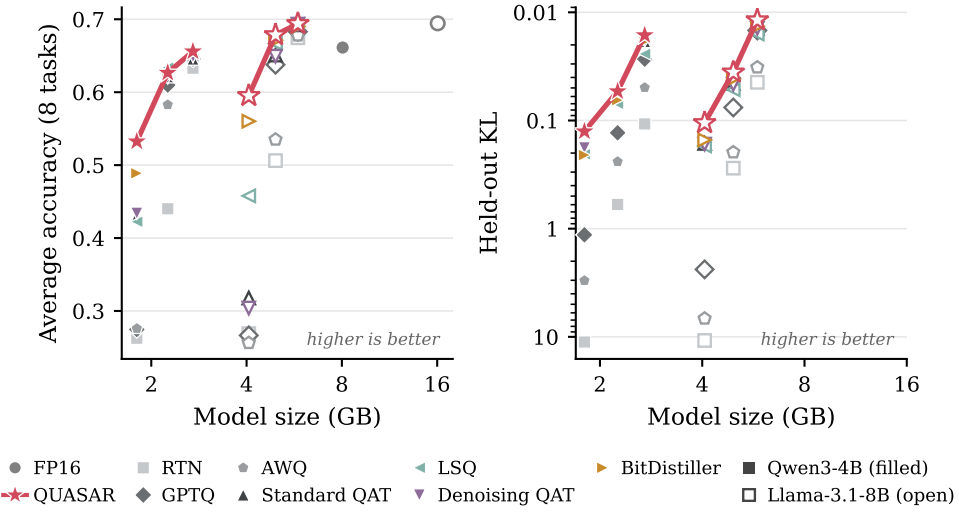


Figure 18: Average accuracy over eight benchmarks (left) and KL to the full-precision model on held-out data (right), versus model size on disk, for the INT4/INT3/INT2 checkpoints of Qwen3-4B-Thinking-2507 (filled markers) and Llama-3.1-8B-Instruct (open markers). Gray circles mark the full-precision models.

Table 8: Net preference for QUASAR in the pairwise LLM-judge study of Figure 6: for each opponent and model, the mean per-prompt preference in $[-1, 1]$ (+1 means QUASAR is preferred on every prompt), with a 95% bootstrap confidence interval over the 128 prompts. The final row compares QUASAR with the full-precision model.

vs. QUASAR	Qwen3-4B-Thinking-2507	Llama-3.1-8B-Instruct
RTN	+1.00 [+1.00, +1.00]	+0.98 [+0.96, +1.00]
GPTQ	+1.00 [+1.00, +1.00]	+0.97 [+0.94, +0.99]
AWQ	+1.00 [+1.00, +1.00]	+0.99 [+0.97, +1.00]
Standard QAT	+0.32 [+0.21, +0.42]	+0.97 [+0.94, +0.99]
LSQ	+0.27 [+0.15, +0.38]	+0.37 [+0.25, +0.49]
Denoising QAT	+0.31 [+0.21, +0.41]	+0.98 [+0.95, +1.00]
BitDistiller	+0.24 [+0.12, +0.35]	+0.29 [+0.17, +0.41]
FP16	-0.30 [-0.40, -0.20]	-0.46 [-0.56, -0.37]

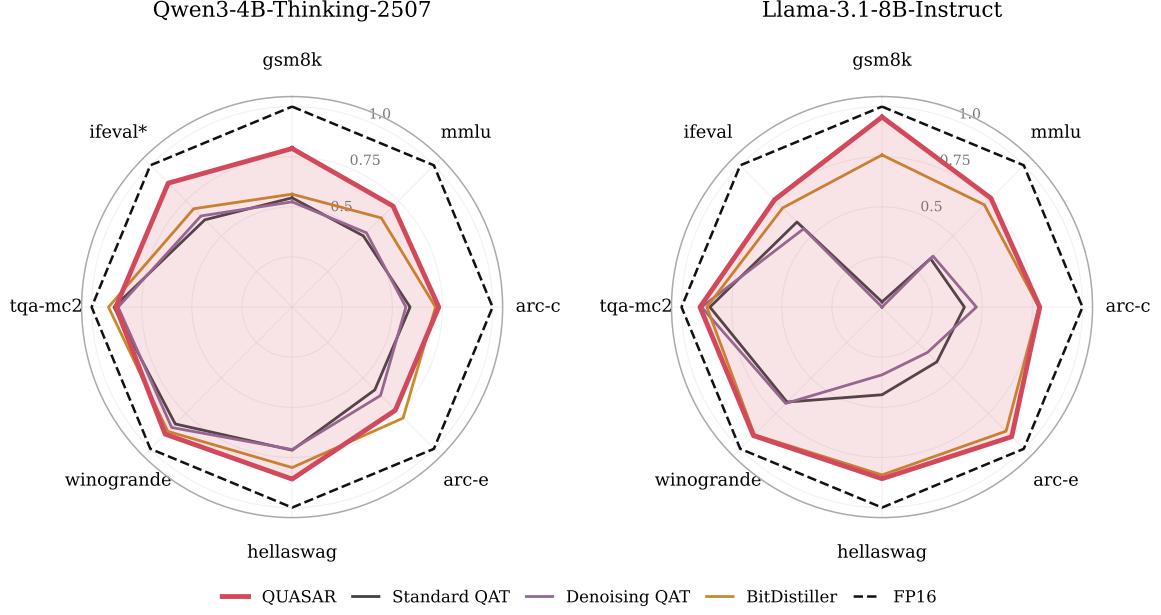


Figure 19: Per-task accuracy of the INT2 QAD checkpoints of Qwen3-4B-Thinking-2507 (left) and Llama-3.1-8B-Instruct (right) on the eight benchmarks of Table 1 and 2, each task normalized by the full-precision model’s accuracy.

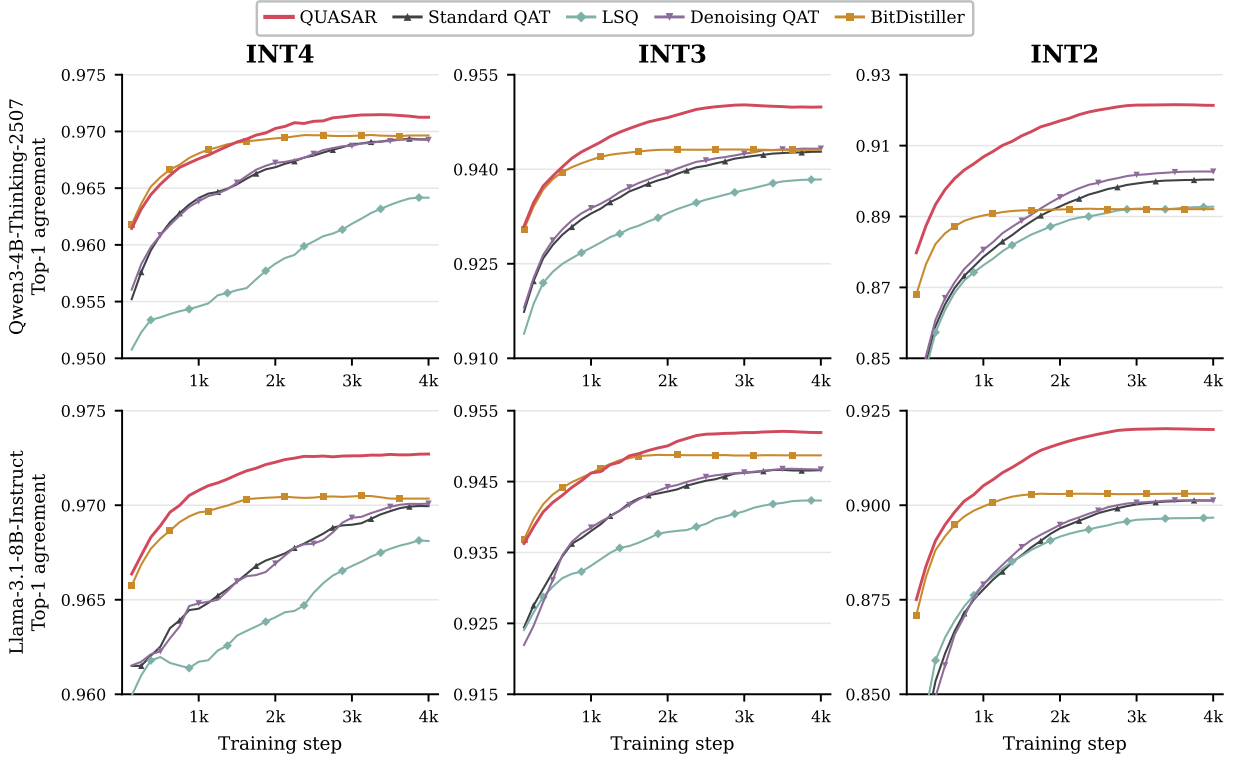


Figure 20: Top-1 agreement with the full-precision model on held-out data during QAD of Qwen3-4B-Thinking-2507 (top row) and Llama-3.1-8B-Instruct (bottom row) at INT4, INT3, and INT2, for the QAT methods of Figure 4 and 5.

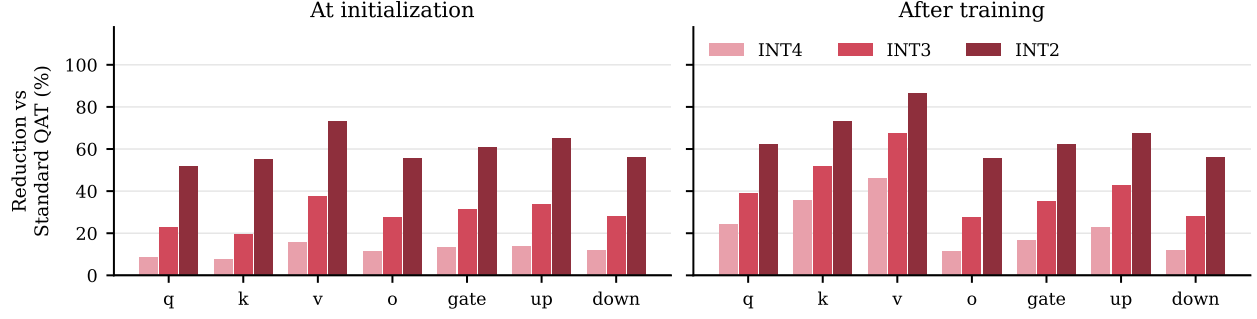


Figure 21: Reduction of the loss-aware reconstruction error achieved by QUASAR relative to Standard QAT, for Qwen3-4B-Thinking-2507 at INT4, INT3, and INT2, split by projection type, at initialization (left) and at the end of training (right). Bars show the median reduction over the modules of each projection type, with the error measured against the full-precision weights.

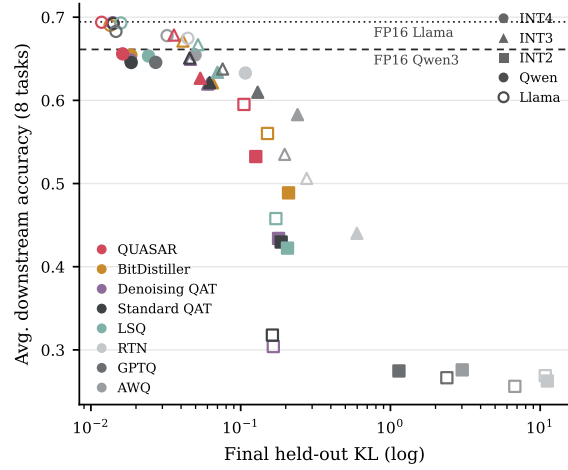


Figure 22: Average accuracy over eight benchmarks versus final KL to the full-precision model on held-out data, for the QAT and PTQ methods of Table 1 and 2 at INT4, INT3, and INT2 on both healing models. Dashed lines mark the full-precision models.

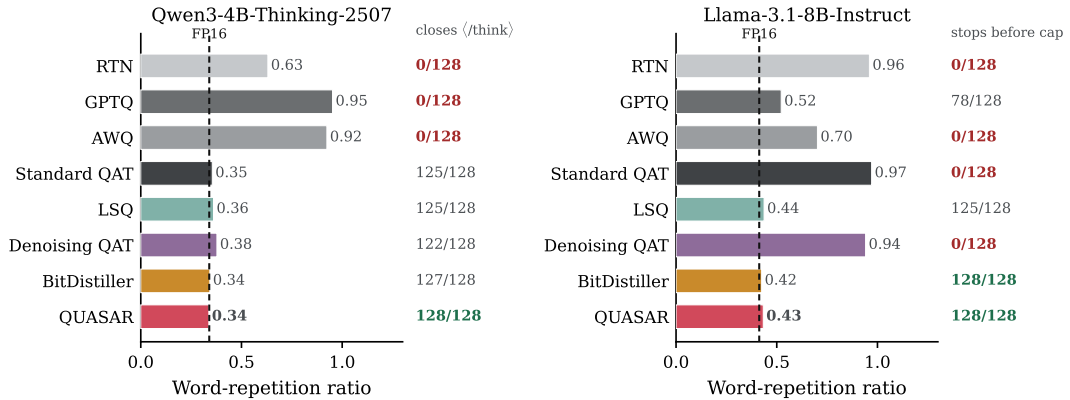


Figure 23: Response statistics of the INT2 checkpoints of Qwen3-4B-Thinking-2507 (left) and Llama-3.1-8B-Instruct (right) on 128 held-out prompts per model, with greedy decoding. Bars: the share of words in a response that repeat earlier words, averaged over prompts (dashed line: full-precision model). Counts on the right of each panel: how many of the 128 responses terminate, meaning Qwen closes its reasoning block and Llama stops before the token cap.

Prompt: <i>Write a story about a man being stranded on an island.</i>	
FP16	The salt stung Leo’s eyes before he even registered the crash. One moment, he was shouting into the cockpit of his battered sedan, the next, the world...
<i>2-bit (INT2)</i>	
RTN	max o mil MD o. over md edge md mdish ai } ats } } further display oscs <igrated found "arql eventually o, same, if (cs1) < <o <ql <, \ql o < <! (< ...
GPTQ	*A [R] :Help "l" (or: "a" in 1000+). 0 * F : 0 (To the 0) : 0 : 0 : 0 : 0 : 0 : 0 : 0 : 0 : 0 : 10000: 0 : 10000: 0 : 0 : 0 : 0 : 0 : 0 : 0 : 0 : 0 : ...
AWQ	go {r=0 {r=1, 100 100 100 100oREENo.1 oo100ooo 100o 10 100. 100 10 10 1 10 1 1 1 0 1 100 1 1 1 1 1 (1 o 1 2 1 02 1 1 1 1 1 1 1. 1 1 1 1 1 1 1 1 1 1 1 ...
Standard QAT	The first thing Elias noticed wasn’t the ocean, but the *absence* of it. Not the roar of waves, but the silence of his own thoughts, a hollow space wh...
Denosing QAT	Leo woke to the sound of water. Not the gentle lapping of a tide, but a relentless, crashing roar against the rocks of a tiny, rocky island he’d never...
LSQ	Leo woke to the smell of salt and wet earth. The first thing he noticed wasn’t the storm, but the *absence* of sound. The roar of the ocean that had b...
BitDistiller	Kai’s first thought wasn’t panic, but a slow, heavy dread. The salt air stung his eyes, thick with the scent of wet earth and something else—something...
QUASAR (ours)	The last thing Alex remembered was the *thud* of the car door slamming shut, the roar of the storm tearing through the windshield, and the sickening l...
<i>3-bit (INT3)</i>	
RTN	Okay, the user wants a story about a man stranded on an island. Hmm, let’s unpack that. First, I should consider what they might need. Are they writin...
GPTQ	The sea had taken him. Not with a crash, but with a slow, suffocating pull. Elias had been a fisherman, not a sailor, and the *Mariana*—his small, stu...
AWQ	The storm had been a fury of glass and thunder, tearing the sea from its bones. Elias woke to the *crack* of splintered wood and the sharp, metallic s...
Standard QAT	Elias woke to the sound of the ocean, not the crash of waves, but the *thump-thump-thump* of his own heart against his ribs. Salt stung his eyes, and ...
Denosing QAT	Leo woke to the smell of salt and something else – wet earth, maybe, or decaying leaves. His head throbbed, a dull ache behind his eyes. He was lying ...
LSQ	## The Island That Forgot to Breathe The last thing Elias remembered was the *thump* of the boat hitting the water, the sickening lurch of the world s...
BitDistiller	Leo’s last memory was the *thwack* of a wave against the hull of the *Marlin*, the sharp sting of salt in his eyes, and the terrifying, silent *thud* ...
QUASAR (ours)	Leo’s last memory was the *scream* of the engine, the sickening lurch as the boat plunged into the churning grey sea, and the world dissolving into a ...
<i>4-bit (INT4)</i>	
RTN	Leo didn’t know how he got there. One moment he was a man in a sleek, unremarkable boat, the next—*thud*—a violent wave slammed the hull, the engine d...
GPTQ	Elias’s world dissolved in a roar. One moment, he was gripping the steering wheel of his aging sedan, the rain lashing the windshield like a thousand ...
AWQ	Leo’s last thought before the world dissolved into water and silence was the *thump* of the boat’s hull against the rocks. Then, the roar of the ocean...
Standard QAT	Leo didn’t know how long he’d been adrift when the first wave of panic hit. It wasn’t the cold, the salt, or the crushing weight of the ocean’s indiff...
Denosing QAT	Leo woke to the sound of the ocean, not the roar of a storm, but the low, insistent *thrum* of the sea against the rocks. His head throbbed, a dull ac...
LSQ	Leo’s last memory was the *thud* of the plane hitting the water, not the roar of the engines or the frantic shouts of the crew. He woke up gasping, th...
BitDistiller	The crash wasn’t loud. It was a sickening *thud* of metal and earth, followed by a silence so profound it felt like the island itself had held its bre...
QUASAR (ours)	The impact wasn’t a crash. It was a *sigh* of the world ending. Ben’s world dissolved into a sickening lurch, the roar of the ocean swallowing his scr...

Figure 24: The beginning of each method’s response to one held-out prompt, for Qwen3-4B-Thinking-2507 at INT2, INT3, and INT4 with greedy decoding.

D Additional Adaptation Results

This appendix extends the adaptation results of Section 6.3 with three analyses of the INT2 checkpoints: how each model’s generated samples split into answer outcomes, how dispersed its sampled successes are, and how divergence from the full-precision SFT model evolves along a reasoning trace.

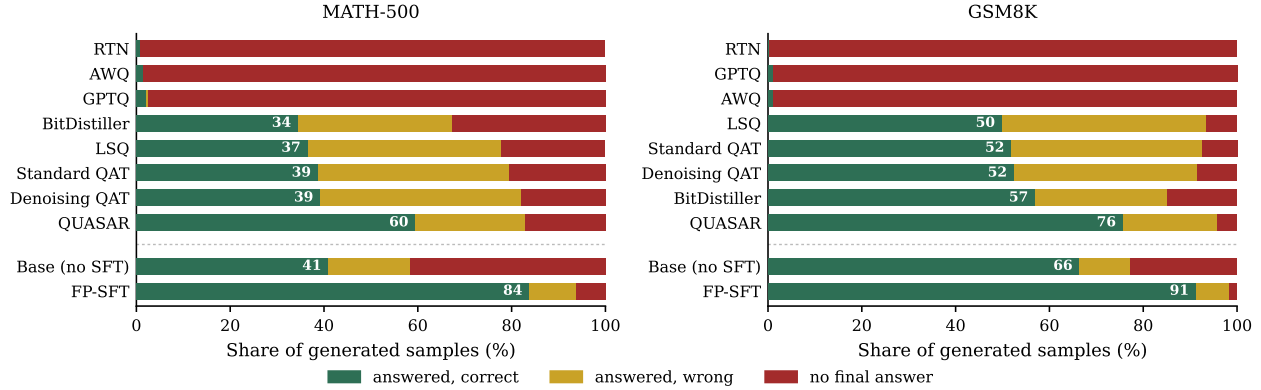


Figure 25: Ratios of correct answers, wrong answers, and missing final answers across all samples generated by the INT2 PTQ and QAT checkpoints of Qwen3-4B-Base, on MATH-500 and GSM8K. The bottom two rows (un-finetuned base model and full-precision SFT) are included for reference.

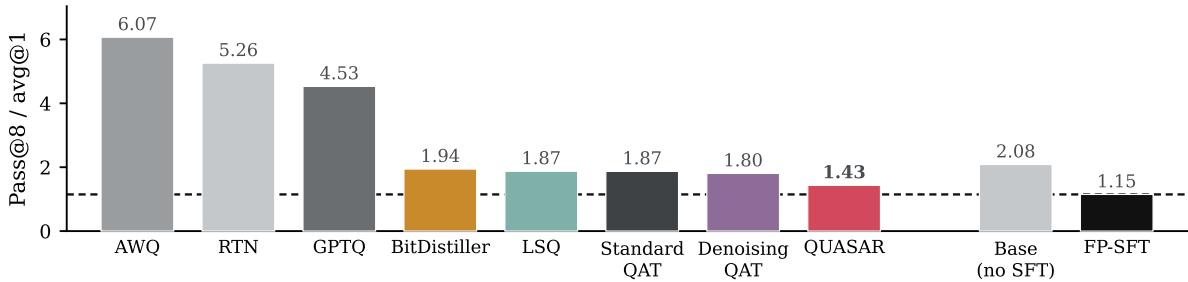


Figure 26: How consistent each INT2 checkpoint of Qwen3-4B-Base is across repeated attempts: the ratio of pass@8 to avg@1 on MATH-500, with eight samples per problem at temperature 0.6. Avg@1 is the accuracy of a single attempt; pass@8 is the share of problems solved at least once in eight attempts. A ratio of 1 means the model solves the same problems on every attempt; a larger ratio means its successes are more hit-or-miss. The dashed line marks full-precision SFT; the two rightmost bars are references.

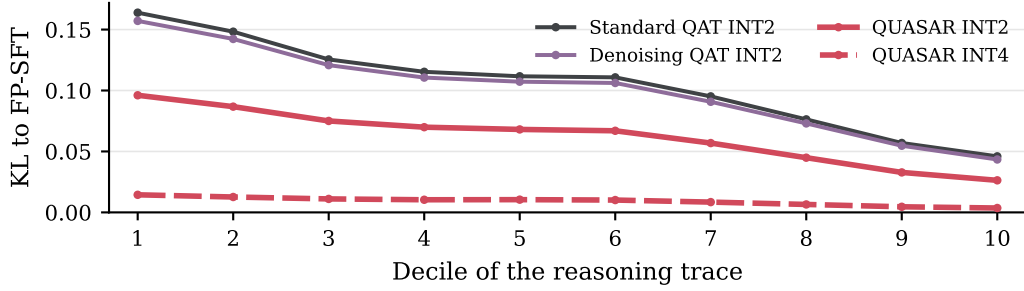


Figure 27: How far each quantized checkpoint drifts from the full-precision SFT model as a reasoning trace unfolds. We take 1,000 traces generated by the full-precision SFT model on MATH-500, feed the same tokens to every model, and measure the per-position forward KL to the full-precision SFT model, averaged within each tenth of the trace.